



ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE
TECHNOLOGIES

DOCUMENTATION TECHNIQUE COMPLÈTE

Kubernetes Enterprise Platform (KEP)

Conception, Déploiement, Sécurisation Zéro-Trust, Observabilité et
Automatisation Intelligente (AIOps)

Auteur :

Farah BEN CHIKHA

Élève Ingénieur

Encadrant :

Pr. Omar FITOURI

Professeur Universitaire

Organisme d'Accueil : **ESPRIT**

Année Universitaire : 2025 - 2026

Table des matières

Introduction Générale	5
1 Architecture Globale de la Plateforme	7
1.1 Objectif	7
1.2 Détail des trois machines virtuelles	7
1.3 Explications techniques	7
1.4 Bilan	8
2 Infrastructure as Code avec Terraform	9
2.1 Objectif	9
2.2 Mise en œuvre	9
2.3 Bilan	10
3 Mise en Place de la Plateforme de Sécurité et Supervision ('elk-vault')	11
3.1 Objectif	11
3.2 Mise en œuvre	11
3.3 Explications techniques	13
3.4 Bilan	14
4 Mise en Place du Cluster Kubernetes	15
4.1 Objectif	15
4.2 Présentation des composants de base	15
4.3 Mise en œuvre	16
4.3.1 Étape 1 — Script d'installation et de préparation système (<code>install_k8s.sh</code>)	16
4.3.2 Étape 2 — Initialisation du Control Plane sur le Master	17
4.3.3 Étape 3 — Déploiement du réseau de pods Calico CNI	18
4.3.4 Étape 4 — Jonction du nœud Worker au cluster	18
4.4 Explications techniques	18
4.5 Bilan	19
5 Démonstration et Validation du Socle Technique	20
5.1 Objectif	20

5.2	Mise en œuvre du Runbook de Validation	20
5.2.1	Étape 1 — Validation du Cluster Kubernetes (<code>k8s-master</code>)	20
5.2.2	Étape 2 — Validation de la Pile Observabilité ELK (<code>elk-vault</code>)	21
5.2.3	Étape 3 — Validation de la Sécurité Zéro-Trust (<code>elk-vault</code>)	22
5.3	Explications techniques	22
5.4	Bilan de la Démonstration	23
6	Automatisation de la Configuration avec Ansible	24
6.1	Objectif	24
6.2	Structure du projet et Gestion de l’Inventaire	24
6.2.1	Fichier d’inventaire (<code>hosts.ini</code>)	24
6.2.2	Sécurisation des identifiants via Ansible Vault	25
6.2.3	Protection des accès SSH et privilèges via ‘ <code>secrets.yml</code> ’	25
6.3	Mise en œuvre des Playbooks d’Automatisation	26
6.3.1	Playbook de configuration du Cluster Kubernetes (<code>playbook-k8s.yml</code>)	26
6.3.2	Playbook de la pile ELK & HashiCorp Vault (<code>playbook-elk.yml</code>)	28
6.4	Explications techniques & Idempotence	29
6.5	Bilan	29
7	Intégration Applicative et Sécurité Zéro-Trust (HashiCorp Vault)	30
7.1	Objectif	30
7.2	Principes de l’Injection par Conteneur Sidecar	30
7.3	Mise en œuvre	31
7.3.1	Étape 1 — Déploiement du Vault Agent Injector via Helm	31
7.3.2	Étape 2 — Configuration du Compte de Service et Autorisations (<code>vault-auth.yml</code>)	32
7.3.3	Étape 3 — Extraction des clés de sécurité (Jeton JWT et Certificat CA)	33
7.3.4	Étape 4 — Configuration de la relation de confiance sur Vault (<code>elk-vault</code>)	33
7.3.5	Étape 5 — Résolution du Routage Réseau Virtuel (VMware / Calico)	34
7.3.6	Étape 6 — Déploiement et Test d’Injection (<code>vault-demo.yml</code>)	34
7.4	Explications techniques	36
7.5	Bilan et Preuve d’Exécution	36
8	Observabilité et Centralisation des Logs (ELK Stack)	37
8.1	Objectif	37
8.2	Présentation des Agents d’Observabilité	37
8.2.1	Filebeat — L’Expéditeur de Journaux (Log Shipper)	37
8.2.2	Metricbeat — Le Collecteur de Métriques Systèmes	37

8.3	Mise en œuvre	38
8.3.1	Étape 1 — Ajout du Dépôt Helm d'Elastic	38
8.3.2	Étape 2 — Configuration et Déploiement de Filebeat (<code>filebeat-values.yaml</code>)	38
8.3.3	Étape 3 — Résolution du Dépannage 'ContainerCreating' (Secrets Fictifs)	39
8.3.4	Étape 4 — Configuration et Déploiement de Metricbeat (<code>metricbeat-values.yaml</code>)	39
8.4	Explications techniques & Validation des Flux	40
8.5	Bilan et État d'Exploitation Global	40
9	Containerisation et Déploiement Applicatif WokMaster	42
9.1	Objectif	42
9.2	Mise en œuvre des Builds Multi-étapes (Multi-Stage Builds)	42
9.2.1	Containerisation du Backend Spring Boot (<code>Dockerfile</code>)	42
9.2.2	Containerisation du Frontend Angular (<code>Dockerfile</code>)	43
9.2.3	Déploiement sur le Master Kubernetes	43
9.3	Expositions et Test d'Accès Réseau	44
9.4	Flux de Mise à Jour Développeur	44
9.5	Bilan	44
10	Sauvegarde et Restauration de la Base de Données MySQL	45
10.1	Objectif	45
10.2	Mise en œuvre	45
10.2.1	Script de Sauvegarde Automatisée (<code>backup-db.sh</code>)	45
10.2.2	Script de Restauration (<code>restore-db.sh</code>)	46
10.3	Explications techniques	46
10.4	Bilan	46
11	Diagnostic Intelligent avec K8sGPT et Google Gemini (AIOps)	47
11.1	Objectif	47
11.2	Mise en œuvre	47
11.2.1	Étape 1 — Installation de K8sGPT v0.3.40	47
11.2.2	Étape 2 — Configuration du Fournisseur IA Google Gemini	47
11.2.3	Étape 3 — Simulation de Panne et Diagnostic IA	48
11.3	Bilan	48
12	Alerting Temps Réel avec Kubewatch et Discord	49
12.1	Objectif	49
12.2	Mise en œuvre	49
12.2.1	Étape 1 — Configuration du Webhook Discord et Résolution DNS CoreDNS	49

12.2.2	Étape 2 — Déploiement de Kubewatch (<code>kubewatch.yaml</code>)	49
12.2.3	Étape 3 — Test de Réception d'Alerte en Direct	50
12.3	Bilan	51
13	Déploiement Continu et GitOps avec ArgoCD	52
13.1	Objectif	52
13.2	Mise en œuvre	52
13.2.1	Étape 1 — Déploiement d'ArgoCD via Server-Side Apply	52
13.2.2	Étape 2 — Exposition Web NodePort et Récupération du Mot de Passe	53
13.2.3	Étape 3 — Création de la Liaison GitOps (<code>argocd-app.yaml</code>) . . .	54
13.3	Explications techniques	56
13.4	Bilan	56
14	Auto-guérison AIOps avec l'Agent MCP Custom (Model Context Pro- tocol)	57
14.1	Objectif	57
14.2	Architecture et Protocole MCP	57
14.3	Mise en œuvre	57
14.3.1	Étape 1 — Code du Serveur MCP (<code>mcp_k8s_server.py</code>)	57
14.3.2	Étape 2 — Code de l'Agent Client Gemini MCP (<code>mcp_agent_run.py</code>)	60
14.3.3	Démonstration de la Boucle d'Auto-guérison en Direct	61
14.4	Explications techniques	62
14.5	Bilan	63
15	Synthèse et Gestion de Projet (Sprints Agile sur 8 Semaines)	64
15.1	Objectif	64
15.2	Planification des Sprints (Chrono-Stage sur 8 Semaines)	64
15.3	Matrice de Synthèse d'Intégration	65
15.4	Bilan de l'Intégration	66
	Conclusion Générale et Perspectives	67
A	Annexes	68
A.1	Annexe A : Fichier d'inventaire Ansible (' <code>hosts.ini</code> ')	68
A.2	Annexe B : Code du Serveur MCP (' <code>mcp_k8s_server.py</code> ')	68
	Webographie et Ressources Techniques	69

Introduction Générale

Contexte du projet

Ce document constitue la documentation technique de référence du projet **Kubernetes Enterprise Platform (KEP)**. Il retrace, de manière exhaustive et méthodique, l'ensemble des travaux menés pour concevoir, déployer, sécuriser, superviser et automatiser une plateforme d'hébergement d'entreprise basée sur **Kubernetes**, destinée à héberger l'application métier **WokMaster** (backend Spring Boot, frontend Angular, persistance MySQL).

L'objectif dépasse le simple déploiement applicatif : il s'agit de construire un écosystème DevOps et DevSecOps complet, industrialisé et reproductible, intégrant :

- Le **provisioning d'infrastructure** par Infrastructure as Code (Terraform) ;
- La **configuration automatisée** des systèmes (Ansible Vault) ;
- L'**orchestration de conteneurs** (Kubernetes & containerd) ;
- La **gestion centralisée des secrets** (HashiCorp Vault) selon une approche Zéro-Trust ;
- L'**observabilité complète** (logs et métriques) via la pile ELK, Filebeat et Metricbeat ;
- Le **déploiement continu** basé sur GitOps (ArgoCD) ;
- L'**alerting temps réel** (Kubewatch sur Discord) ;
- Et l'intégration d'**agents d'intelligence artificielle (AIOps)** pour le diagnostic (K8sGPT) et l'auto-guérison guidée via le protocole **Model Context Protocol (MCP)** et **Google Gemini**.

Objectifs de la plateforme

1. **Haute disponibilité et scalabilité** grâce à l'orchestration Kubernetes.
2. **Sécurité renforcée** par l'injection dynamique de secrets en mémoire vive RAM (`tmpfs`).
3. **Observabilité totale** : centralisation des journaux et des métriques sur Kibana.

4. **Automatisation intégrale** : du provisionnement au déploiement sans action manuelle.
5. **Résilience intelligente (AIOps)** : capacité d'auto-diagnostic et d'auto-remédiation assistée.

Méthodologie du document

Chaque chapitre suit la même structure pédagogique : **Objectif** → **Mise en œuvre** → **Explications techniques** → **Captures / Preuves** → **Bilan**. Par sécurité, les secrets sensibles (jetons Vault, clés Shamir, clés API) sont remplacés par des placeholders génériques (ex : <VAULT_ROOT_TOKEN>).

Chapitre 1

Architecture Globale de la Plateforme

1.1 Objectif

Présenter la structure matérielle et réseau de la plateforme KEP, basée sur trois machines virtuelles isolées sous VMware Workstation Pro.

1.2 Détail des trois machines virtuelles

- **VM 1 — k8s-master (192.168.233.188 - 2 vCPU / 2.3 Go RAM) :** Héberge le Control Plane Kubernetes (API Server, Scheduler, etcd), ArgoCD, Kubewatch, K8sGPT et le serveur MCP / Agent Python Gemini.
- **VM 2 — k8s-worker (192.168.233.190 - 2 vCPU / 4 Go RAM) :** Exécute les conteneurs WokMaster (Frontend Angular NodePort 30081, Backend Spring Boot), MySQL 8.0, le sidecar Vault Agent (tmpfs RAM) et les agents Filebeat/Metricbeat.
- **VM 3 — elk-vault (192.168.233.189 - 2 vCPU / 3 Go RAM) :** Serveur externe dédié à la sécurité Zéro-Trust (HashiCorp Vault) et à l'observabilité (Elasticsearch 9200, Kibana 5601).

1.3 Explications techniques

La séparation du serveur de sécurité et de supervision (**elk-vault**) en dehors du cluster Kubernetes répond aux principes de défense en profondeur (*defense in depth*). Même en cas de compromission totale du cluster applicatif, les clés de chiffrement Vault et les journaux d'audit centralisés restent isolés et protégés.

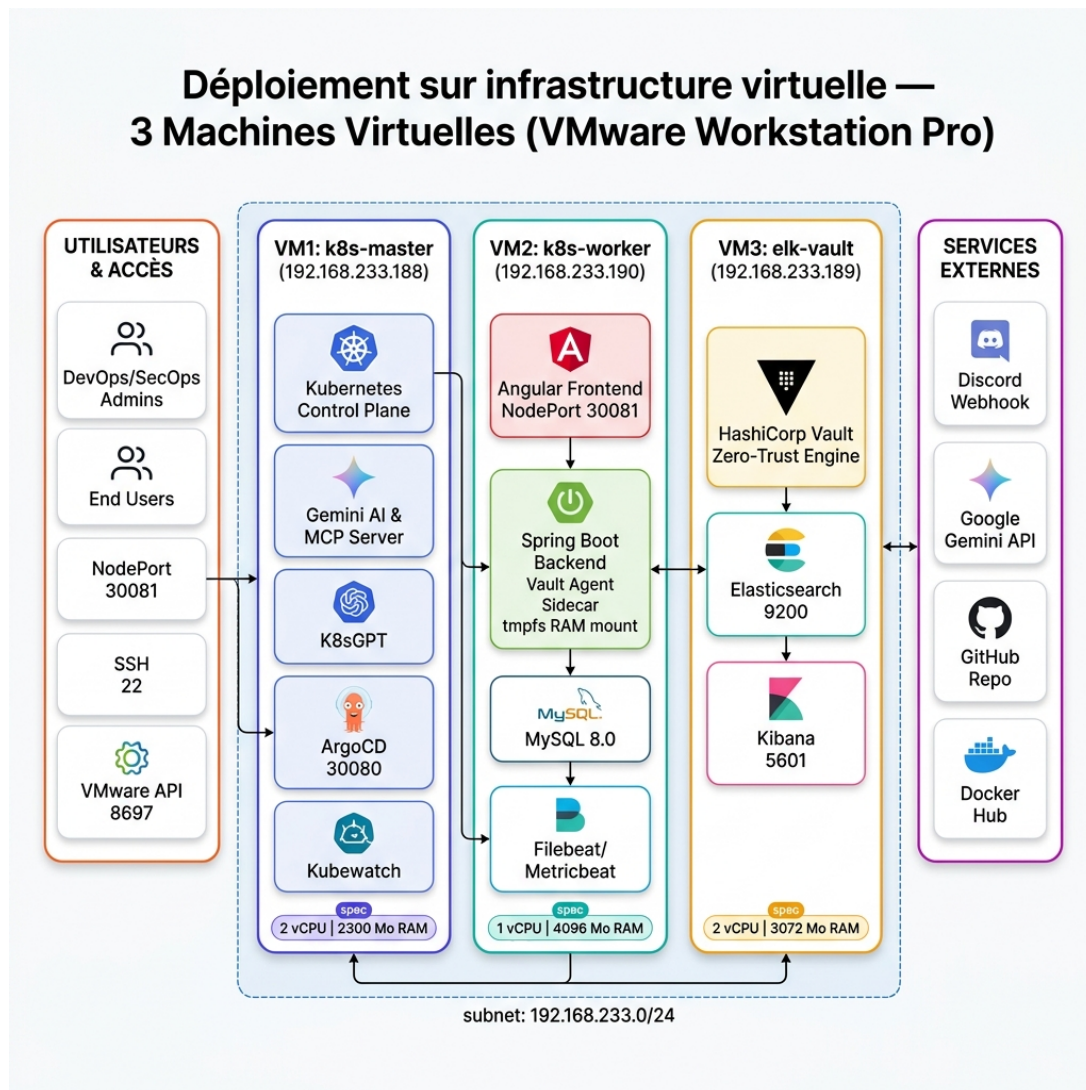


FIGURE 1.1 – Schéma d’architecture physique et applicative de la plateforme KEP avec le dimensionnement réel des VMs

1.4 Bilan

L’architecture à trois nœuds établit un cloisonnement étanche entre le contrôle, l’exécution et la sécurité.

Chapitre 2

Infrastructure as Code avec Terraform

2.1 Objectif

Automatiser le provisionnement des trois machines virtuelles sous VMware Workstation Pro via Terraform et l'API REST local (`vmrest.exe`).

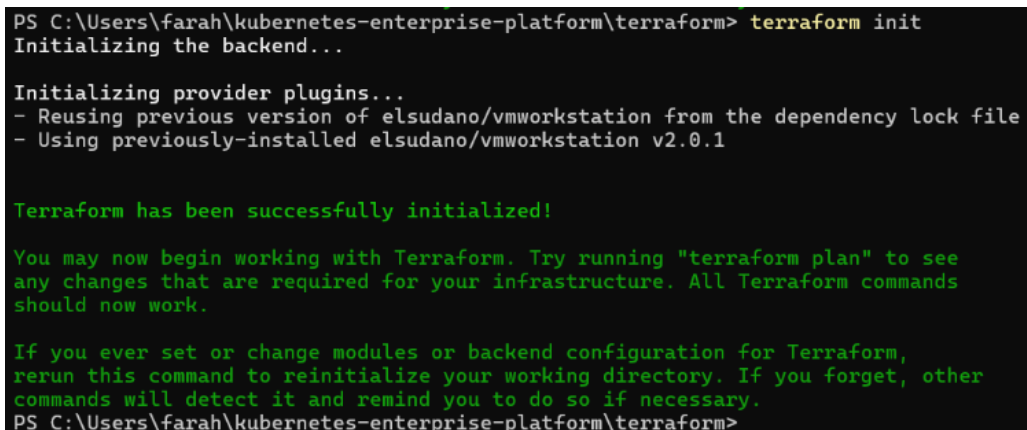
2.2 Mise en œuvre

1. Démarrage du service d'API REST VMware sur l'hôte Windows :

```
1 cd "C:\Program Files (x86)\VMware\VMware Workstation"
2 .\vmrest.exe -p 8697
```

2. Exécution du script PowerShell `get_vms.ps1` pour extraire l'ID de la VM modèle `ubuntu-base`.
3. Lancement du déploiement séquentiel Terraform :

```
1 cd terraform/
2 terraform init
3 terraform apply -parallelism=1
```



```
PS C:\Users\farah\kubernetes-enterprise-platform\terraform> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of elsudano/vmworkstation from the dependency lock file
- Using previously-installed elsudano/vmworkstation v2.0.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\farah\kubernetes-enterprise-platform\terraform>
```

FIGURE 2.1 – Initialisation du fournisseur VMware Workstation et téléchargement des plugins via `terraform init`

```

PS C:\Users\farah\kubernetes-enterprise-platform\terraform> terraform apply -parallelism=1

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# vmworkstation_virtual_machine.elk-vault will be created
+ resource "vmworkstation_virtual_machine" "elk-vault" {
  + denomination = "elk-vault"
  + id           = (known after apply)
  + ip          = "0.0.0.0/0"
  + memory      = 3872
  + path        = "C:\\Users\\farah\\Virtual Machines\\elk-vault\\elk-vault.vmx"
  + processors   = 2
  + sourceid    = "JU1H89H67D6MTF09GQD12VIU2PJI48E"
  + state       = "on"
}

# vmworkstation_virtual_machine.k8s-master will be created
+ resource "vmworkstation_virtual_machine" "k8s-master" {
  + denomination = "k8s-master"
  + id           = (known after apply)
  + ip          = "0.0.0.0/0"
  + memory      = 2048
  + path        = "C:\\Users\\farah\\Virtual Machines\\k8s-master\\k8s-master.vmx"
  + processors   = 2
  + sourceid    = "JU1H89H67D6MTF09GQD12VIU2PJI48E"
  + state       = "on"
}

# vmworkstation_virtual_machine.k8s-worker will be created
+ resource "vmworkstation_virtual_machine" "k8s-worker" {
  + denomination = "k8s-worker"
  + id           = (known after apply)
  + ip          = "0.0.0.0/0"
  + memory      = 2048
  + path        = "C:\\Users\\farah\\Virtual Machines\\k8s-worker\\k8s-worker.vmx"
  + processors   = 1
  + sourceid    = "JU1H89H67D6MTF09GQD12VIU2PJI48E"
  + state       = "on"
}

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

vmworkstation_virtual_machine.k8s-master: Creating...
vmworkstation_virtual_machine.k8s-master: Still creating... [00m10s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m20s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m30s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m40s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m50s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m00s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m10s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m20s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m30s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m40s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m50s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [02m00s elapsed]

```

FIGURE 2.2 – Provisionnement séquentiel des 3 machines virtuelles via `terraform apply -parallelism=1`

2.3 Bilan

Les trois VMs sont provisionnées en moins de 3 minutes avec leurs adresses IP respectives.

Chapitre 3

Mise en Place de la Plateforme de Sécurité et Supervision ('elk-vault')

3.1 Objectif

Installer et valider manuellement la pile ELK (Elasticsearch 8.x, Kibana) et HashiCorp Vault sur la VM 192.168.233.189 avant d'automatiser le processus via Ansible.

3.2 Mise en œuvre

1. Renommage du nom d'hôte de la machine virtuelle :

```
1 sudo hostnamectl set-hostname elk-vault
```

2. Exécution du script d'installation automatisé `install_elk.sh` :

```
1 ssh farah@192.168.233.189
2 chmod +x install_elk.sh
3 ./install_elk.sh
```

[illegible]

FIGURE 3.1 – Fin de l'exécution du script `install_elk.sh` et démarrage réussi des services Elasticsearch, Kibana et Vault

3. Validation du démarrage des services d'observabilité :

```
1 # Test de Kibana (Port 5601)
2 curl -I http://localhost:5601
3
4 # Test d'Elasticsearch (Port 9200 HTTPS)
5 curl -k -u elastic:<ELASTIC_PASSWORD> https://localhost:9200
```

```

$ curl -i http://localhost:5001
HTTP/1.1 200 OK
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera=(self), display-capture=(self), fullscreen=(self), geolocation(), microphone(), web-share()
cross-origin-opener-policy: same-origin
content-security-policy: script-src 'report-sample' 'self'; worker-src 'report-sample' 'self' blob; style-src 'report-sample' 'self' 'unsafe-inline'
content-security-policy-report-only: form-action 'report-sample' 'self'; default-src 'report-sample' 'none'; font-src 'report-sample' 'self'; img-src 'report-sample' 'self' data: tiles.maps.elastic.co; connect-src 'report-sample' 'self' telemetry.telemetry.elastic.co telemetry-staging.telemetry.elastic.co tiles.maps.elastic.co vector.maps.elastic.co
kbn-name: elk-vault
content-type: text/html; charset=utf-8
connection: keep-alive, no-cache, no-store, must-revalidate
content-length: 37743
vary: accept-encoding
Date: Tue, 21 Jul 2026 01:04:58 GMT

```

FIGURE 3.2 – Validation du fonctionnement du serveur Web Kibana sur le port 5601 via la réponse HTTP 200 OK

```

farah@elk-vault:~$ curl -k -u elastic:i1*I_3YeN5XP933Wj9Gk https://localhost:9200
{
  "name" : "elk-vault",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "cTBxAZ1BTEunAd-99VkBzA",
  "version" : {
    "number" : "8.19.18",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "e8ac685d1710aae2c9fc9ca61e2956ab9424d5f8",
    "build_date" : "2026-06-26T10:09:47.981719133Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}

```

FIGURE 3.3 – Validation de la connexion chiffrée HTTPS et de l'authentification de l'utilisateur `elastic` sur Elasticsearch 8.x

4. Initialisation et déverrouillage de HashiCorp Vault :

```

1 export VAULT_SKIP_VERIFY=true
2 vault status
3 vault operator init
4 vault operator unseal <UNSEAL_KEY_1>
5 vault operator unseal <UNSEAL_KEY_2>
6 vault operator unseal <UNSEAL_KEY_3>

```

3.3 Explications techniques

Par conception, HashiCorp Vault démarre toujours à l'état `Sealed: true`. L'algorithme de partage de secret de Shamir exige la saisie d'au moins 3 clés indépendantes sur 5 pour reconstituer la clé maîtresse en mémoire vive RAM sans la stocker sur disque.

```

farah@elk-vault:~$ vault operator unseal
WARNING! VAULT_ADDR and -address unset. Defaulting to https://127.0.0.1:8200.
Unseal Key (will be hidden):
Key          Value
---          -
Seal Type    shamir
Initialized  true
Sealed       false
Total Shares 5
Threshold    3
Version      2.0.3
Build Date   2026-06-17T12:39:45Z
Storage Type  file
Cluster Name  vault-cluster-481a2363
Cluster ID    743fa875-16a8-24c5-8edd-7cc09fe1bc4c
HA Enabled    false
farah@elk-vault:~$

```

FIGURE 3.4 – Déverrouillage réussi du coffre-fort HashiCorp Vault à l'aide des clés de Shamir (`Sealed: false`)

3.4 Bilan

La VM d'observabilité est active et prête à recevoir les métriques et sécuriser les secrets du cluster. —

Chapitre 4

Mise en Place du Cluster Kubernetes

4.1 Objectif

Déployer et configurer un cluster Kubernetes bi-nœuds (`k8s-master` et `k8s-worker`) à l'aide du script d'automatisation `install_k8s.sh`, de l'outil d'initialisation `kubeadm`, du moteur d'exécution `containerd` et du réseau de pods Calico CNI.

4.2 Présentation des composants de base

Chaque nœud du cluster s'appuie sur quatre briques logicielles fondamentales :

- `containerd` : Le moteur d'exécution des conteneurs (*Container Runtime*). Il est chargé de télécharger les images de conteneurs et de les exécuter physiquement sur le système d'exploitation.
- `kubeadm` : L'outil d'installation et d'initialisation du cluster. Il permet d'initialiser le Control Plane sur le Master (`kubeadm init`) et de lier le nœud Worker au cluster (`kubeadm join`).
- `kubelet` : L'agent principal de Kubernetes s'exécutant en tâche de fond sur chaque nœud. Il reçoit les instructions du Master, donne l'ordre à `containerd` de lancer les conteneurs et surveille leur état en continu.
- `kubectl` : L'interface en ligne de commande (CLI) d'administration. Elle permet à l'administrateur de communiquer directement avec l'API Server du Master.

4.3 Mise en œuvre

4.3.1 Étape 1 — Script d'installation et de préparation système (install_k8s.sh)

Pour garantir une configuration identique et reproductible sur le Master (192.168.233.188) et le Worker (192.168.233.190), le script Shell automatisé suivant a été rédigé et exécuté :

```
1 #!/bin/bash
2 set -e
3
4 echo "=== 1. Desactivation permanente du swap (Exigence K8s) ==="
5 sudo swapoff -a
6 sudo sed -i '/ swap / s/^(\.*\)$/#1/g' /etc/fstab
7
8 echo "=== 2. Configuration des modules reseau du noyau ==="
9 cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
10 overlay
11 br_netfilter
12 EOF
13
14 sudo modprobe overlay
15 sudo modprobe br_netfilter
16
17 # Configuration sysctl requise pour le reseau de pontage (bridge)
18 cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
19 net.bridge.bridge-nf-call-iptables = 1
20 net.bridge.bridge-nf-call-ip6tables = 1
21 net.ipv4.ip_forward = 1
22 EOF
23
24 sudo sysctl --system
25
26 echo "=== 3. Installation du runtime de conteneurs containerd ==="
27 sudo apt-get update && sudo apt-get install -y ca-certificates curl
28 gnupg lsb-release
29 sudo mkdir -p /etc/apt/keyrings
30 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
31 dearmor -o /etc/apt/keyrings/docker.gpg
32 echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings
33 /docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -
34 cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/
35 null
36
37 sudo apt-get update && sudo apt-get install -y containerd.io
```

```

33
34 # Configuration correcte de SystemdCgroup pour containerd (exigence K8s)
35 sudo mkdir -p /etc/containerd
36 containerd config default | sudo sed 's/SystemdCgroup = false/
    SystemdCgroup = true/g' | sudo tee /etc/containerd/config.toml
37 sudo systemctl restart containerd
38
39 echo "=== 4. Installation des outils Kubernetes (kubelet, kubeadm,
    kubect1) ==="
40 # Ajout de la cle GPG pour les depots Kubernetes v1.30
41 curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key |
    sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
42 # Ajout du depot Kubernetes APT
43 echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https
    ://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/
    sources.list.d/kubernetes.list > /dev/null
44
45 sudo apt-get update
46 sudo apt-get install -y kubelet kubeadm kubect1
47 # On bloque les mises a jour automatiques des outils K8s pour eviter des
    incompatibilites
48 sudo apt-mark hold kubelet kubeadm kubect1
49 sudo systemctl enable --now kubelet
50
51 echo "=== Installation de Kubernetes terminee avec succes ! ==="

```

Listing 4.1 – Script complet de préparation système et d’installation Kubernetes (install_k8s.sh)

L’exécution de ce script (`sudo ./install_k8s.sh`) configure les prérequis du noyau Linux, initialise `containerd` avec `SystemdCgroup=true` et installe les outils Kubernetes verrouillés en version 1.30.

4.3.2 Étape 2 — Initialisation du Control Plane sur le Master

Sur la machine `k8s-master` (192.168.233.188) :

```

1 sudo kubeadm init \
2   --pod-network-cidr=192.168.0.0/16 \
3   --apiserver-advertise-address=192.168.233.188

```

Configuration des droits d’administration de `kubect1` pour l’utilisateur courant :

```

1 mkdir -p $HOME/.kube
2 sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
3 sudo chown $(id -u):$(id -g) $HOME/.kube/config

```

4.3.3 Étape 3 — Déploiement du réseau de pods Calico CNI

À ce stade, le nœud Master apparaît au statut `NotReady` car le réseau de conteneurs n'est pas encore installé. Le plugin réseau `Calico CNI` est déployé pour créer la superposition de réseau virtuel :

```
1 kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/
   v3.26.1/manifests/calico.yaml
```

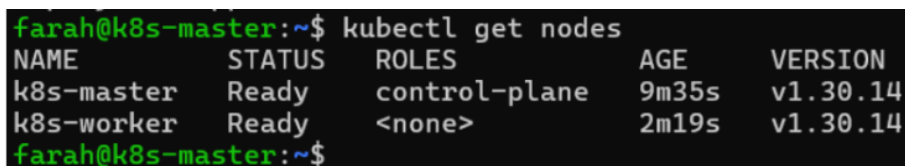
4.3.4 Étape 4 — Jonction du nœud Worker au cluster

Sur le nœud `k8s-worker` (192.168.233.190), la commande de jonction générée par le Master est exécutée :

```
1 sudo kubeadm join 192.168.233.188:6443 \
2   --token <KUBEADM_TOKEN> \
3   --discovery-token-ca-cert-hash sha256:<DISCOVERY_HASH>
```

4.4 Explications techniques

- **Désactivation du Swap** : Kubernetes refuse de démarrer si le swap est actif car le `kubelet` exige un contrôle absolu sur l'allocation de la mémoire vive pour garantir les seuils de Qualité de Service (QoS).
- **Modules Kernel overlay et br_netfilter** : `overlay` permet de superposer les couches de système de fichiers des images Docker, tandis que `br_netfilter` permet au pare-feu d'Ubuntu (`iptables`) de filtrer et router le trafic inter-conteneurs qui transite par les ponts réseau virtuels.
- **Options kubeadm init** : `-pod-network-cidr=192.168.0.0/16` définit la plage d'adresses virtuelles privées attribuées aux pods par Calico, et `-apiserver-advertise-address=` indique l'IP physique du Master sur laquelle le Worker doit s'interconnecter.
- `apt-mark hold` : Verrouille la version des paquets pour éviter qu'une mise à jour automatique d'Ubuntu ne rompe la compatibilité du cluster en production.



```
farah@k8s-master:~$ kubectl get nodes
NAME           STATUS    ROLES           AGE      VERSION
k8s-master     Ready     control-plane   9m35s    v1.30.14
k8s-worker     Ready     <none>          2m19s    v1.30.14
farah@k8s-master:~$
```

FIGURE 4.1 – Vérification du statut opérationnel des nœuds Master et Worker (Ready) avec la commande `kubectl get nodes`

```
farah@k8s-master:~$ kubectl get pods -A
NAMESPACE      NAME                                                    READY   STATUS    RESTARTS   AGE
kube-system    calico-kube-controllers-564985c589-b9d7p             1/1     Running   0           7m5s
kube-system    calico-node-dbdmj                                     1/1     Running   0           7m5s
kube-system    calico-node-g59sq                                     1/1     Running   0           6m4s
kube-system    coredns-55cb58b774-krb74                             1/1     Running   0           13m
kube-system    coredns-55cb58b774-s7k8z                             1/1     Running   0           13m
kube-system    etcd-k8s-master                                       1/1     Running   1           14m
kube-system    kube-apiserver-k8s-master                             1/1     Running   1           14m
kube-system    kube-controller-manager-k8s-master                   1/1     Running   1           14m
kube-system    kube-proxy-57d6f                                     1/1     Running   0           6m4s
kube-system    kube-proxy-h7z7l                                     1/1     Running   0           13m
kube-system    kube-scheduler-k8s-master                             1/1     Running   1           14m
farah@k8s-master:~$
```

FIGURE 4.2 – Vérification de l'état d'exécution (Running) de l'ensemble des pods système (Calico CNI, CoreDNS, kube-proxy) avec la commande `kubectl get pods -A`

4.5 Bilan

L'exécution de la commande `kubectl get nodes` sur le Master confirme que les deux nœuds (`k8s-master` et `k8s-worker`) sont au statut ****Ready****, et la commande `kubectl get pods -A` valide le bon fonctionnement de tous les services système (CoreDNS, kube-proxy, Calico).

Chapitre 5

Démonstration et Validation du Socle Technique

5.1 Objectif

Exécuter la procédure standard de validation opérationnelle (*runbook*) pour vérifier le bon fonctionnement, la sécurité et l'interconnexion réseau de l'ensemble des composants du socle technique (Kubernetes, Elasticsearch, Kibana et Vault) avant le déploiement applicatif.

5.2 Mise en œuvre du Runbook de Validation

5.2.1 Étape 1 — Validation du Cluster Kubernetes (k8s-master)

Connexion SSH au nœud Master (192.168.233.188) et exécution des commandes d'inspection des ressources :

```
1 # 1. Vérifier la présence et l'état des machines du cluster
2 kubectl get nodes
3
4 # 2. Montrer que tous les services système (Calico, DNS, proxy)
   fonctionnent
5 kubectl get pods -A
```

Listing 5.1 – Vérification du statut des nœuds et des pods système sur le Master

```

farah@k8s-master:~$ kubectl get nodes
NAME          STATUS    ROLES          AGE      VERSION
k8s-master    Ready     control-plane   9m35s    v1.30.14
k8s-worker    Ready     <none>          2m19s    v1.30.14
farah@k8s-master:~$

```

FIGURE 5.1 – Vérification du statut opérationnel des nœuds Master et Worker (Ready) avec `kubectl get nodes`

```

farah@k8s-master:~$ kubectl get pods -A
NAMESPACE     NAME                                                    READY   STATUS    RESTARTS   AGE
kube-system    calico-kube-controllers-564985c589-b9d7p              1/1     Running   0           7m5
kube-system    calico-node-dbdmj                                      1/1     Running   0           7m5
kube-system    calico-node-g59sq                                      1/1     Running   0           6m4
kube-system    coredns-55cb58b774-kxb74                             1/1     Running   0           13m
kube-system    coredns-55cb58b774-s7k8z                             1/1     Running   0           13m
kube-system    etcd-k8s-master                                       1/1     Running   1           14m
kube-system    kube-apiserver-k8s-master                             1/1     Running   1           14m
kube-system    kube-controller-manager-k8s-master                   1/1     Running   1           14m
kube-system    kube-proxy-57d6f                                      1/1     Running   0           6m4
kube-system    kube-proxy-h7z7l                                      1/1     Running   0           13m
kube-system    kube-scheduler-k8s-master                             1/1     Running   1           14m
farah@k8s-master:~$

```

FIGURE 5.2 – Vérification de l'état d'exécution (Running) de l'ensemble des pods système avec `kubectl get pods -A`

5.2.2 Étape 2 — Validation de la Pile Observabilité ELK (elk-vault)

Connexion SSH au serveur d'observabilité (192.168.233.189) pour tester la réactivité des API Elasticsearch et Kibana :

```

1 # 3. Tester qu'Elasticsearch (HTTPS) r pond et est s curis
2 curl -k -u elastic:<ELASTIC_PASSWORD> https://localhost:9200
3
4 # 4. Tester que l'interface Web Kibana est active sur le port 5601
5 curl -I http://localhost:5601

```

Listing 5.2 – Validation des services d'observabilité Elasticsearch et Kibana

```

farah@elk-vault:~$ curl -k -u elastic:i1*I_3YeN5XP933Wj9Gk https://localhost:9200
{
  "name" : "elk-vault",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "cTBxAZ1BTEunAd-99VkBzA",
  "version" : {
    "number" : "8.19.18",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "e8ac685d1710aae2c9fc9ca61e2956ab9424d5f8",
    "build_date" : "2026-06-26T10:09:47.981719133Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}

```

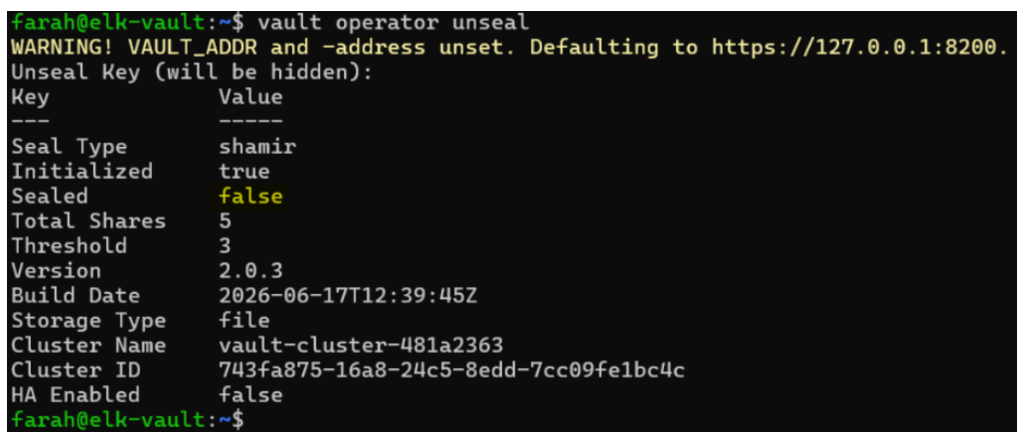
FIGURE 5.3 – Validation de la réponse JSON du moteur Elasticsearch 8.x en HTTPS avec authentification

5.2.3 Étape 3 — Validation de la Sécurité Zéro-Trust (elk-vault)

Vérification de l'état de sécurité du coffre-fort HashiCorp Vault et déverrouillage en direct via le mécanisme de Shamir :

```
1 # 5. Ignorer la v r i f i c a t i o n du certificat auto-sign
2 export VAULT_SKIP_VERIFY=true
3
4 # 6. V r i f i e r l' é t a t de Vault (Sealed: true au d m a r r a g e)
5 vault status
6
7 # 7. D é v e r r o u i l l e r le coffre en direct (ex e c u t é 3 fois avec 3 c l é s
   Shamir d i f f é r e n t e s)
8 vault operator unseal
```

Listing 5.3 – Inspection du statut et déverrouillage de HashiCorp Vault



```
farah@elk-vault:~$ vault operator unseal
WARNING! VAULT_ADDR and -address unset. Defaulting to https://127.0.0.1:8200.
Unseal Key (will be hidden):
Key          Value
---          -
Seal Type    shamir
Initialized  true
Sealed       false
Total Shares  5
Threshold    3
Version      2.0.3
Build Date   2026-06-17T12:39:45Z
Storage Type  file
Cluster Name vault-cluster-481a2363
Cluster ID    743fa875-16a8-24c5-8edd-7cc09fe1bc4c
HA Enabled    false
farah@elk-vault:~$
```

FIGURE 5.4 – Déverrouillage réussi de HashiCorp Vault à l'aide des 3 clés Shamir (Sealed: false)

5.3 Explications techniques

- **Isolation du statut Vault** : Le fait que Vault redémarre systématiquement à l'état `Sealed: true` est un comportement de sécurité **normal et désiré** (*Security by Design*). Les clés master cryptographiques ne sont jamais conservées sur le disque dur.
- **Authentification Elasticsearch 8.x** : La réponse JSON sur le port 9200 confirme que la sécurité HTTPS par défaut et la gestion des rôles utilisateur (*RBAC*) sont actives.

5.4 Bilan de la Démonstration

Ce runbook de validation — reproductible à chaque démarrage de l'environnement — confirme que les 3 piliers fondamentaux (Orchestration Kubernetes, Observabilité ELK, Sécurité Vault) sont 100 % opérationnels et interconnectés sur le réseau virtuel.

Chapitre 6

Automatisation de la Configuration avec Ansible

6.1 Objectif

Industrialiser et automatiser l'intégralité de la configuration logicielle du cluster Kubernetes et de la plateforme ELK / Vault à l'aide de playbooks **Ansible** déclaratifs, chiffrés et idempotents.

L'objectif majeur est de transformer l'ensemble des procédures d'installation et de paramétrage exécutées manuellement dans les Chapitres 3 et 4 (désactivation du swap, configuration du noyau Linux, installation de `containerd`, initialisation Kubernetes, déploiement du réseau Calico, installation d'Elasticsearch, Kibana et Vault) en un processus **100 % automatisé, rejouable et reproductible**.

6.2 Structure du projet et Gestion de l'Inventaire

Les fichiers de configuration Ansible sont regroupés sur le nœud Master (192.168.233.188) dans le répertoire `~/ansible-setup/`.

6.2.1 Fichier d'inventaire (`hosts.ini`)

Le fichier d'inventaire recense l'ensemble des machines virtuelles cibles regroupées par rôles avec leurs adresses IP respectives :

```
1 [master]
2 k8s-master ansible_host=192.168.233.188
3
4 [worker]
5 k8s-worker ansible_host=192.168.233.190
6
7 [elk]
```

```
8 elk-vault ansible_host=192.168.233.189
9
10 [all:vars]
11 ansible_user=farah
12 ansible_ssh_pass="{ vault_ssh_pass }"
13 ansible_become_pass="{ vault_become_pass }"
14 ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Listing 6.1 – Fichier d’inventaire Ansible (hosts.ini)

6.2.2 Sécurisation des identifiants via Ansible Vault

Pour respecter les normes de sécurité SecOps et éviter de stocker les mots de passe SSH et les accès super-utilisateur (sudo) en clair dans le dépôt Git, les variables sensibles sont conservées dans un fichier chiffré par **Ansible Vault** en AES-256 :

```
1 # Chiffrement du fichier de secrets sensibles
2 ansible-vault encrypt secrets.yml
3
4 # Test d'authentification SSH et sudo sur l'ensemble des machines
5 ansible all -m ping -i hosts.ini --ask-vault-pass
```

Listing 6.2 – Chiffrement et gestion des secrets avec Ansible Vault

6.2.3 Protection des accès SSH et privilèges via ‘secrets.yml’

Afin d’éviter de stocker les identifiants de connexion SSH et les mots de passe de privilèges d’administration (sudo) en clair dans le fichier d’inventaire ou le dépôt Git, un fichier de variables dédié nommé **secrets.yml** a été créé :

```
1 # Identifiants de connexion SSH et de privilèges d'administration (sudo)
2 vault_ssh_pass: "farah"
3 vault_become_pass: "farah"
```

Listing 6.3 – Fichier de variables sensibles avant chiffrement (secrets.yml)

Ce fichier est ensuite chiffré avec l’algorithme fort **AES-256** via la commande **ansible-vault** :

```
1 # Chiffrement du fichier de secrets SSH
2 ansible-vault encrypt secrets.yml
3
4 # Pour dater de façon sécurisée le fichier chiffré si besoin :
5 ansible-vault edit secrets.yml
```

Listing 6.4 – Chiffrement du fichier de secrets SSH avec Ansible Vault

Dans le fichier d'inventaire `hosts.ini`, ces variables chiffrées sont automatiquement injectées sous les paramètres `ansible_ssh_pass=" vault_ssh_pass "` et `ansible_become_pass=" vault_become_pass "`.

Lors de chaque exécution d'un playbook, l'administrateur fournit le mot de passe maître de la voûte via l'option `-ask-vault-pass`. Cela garantit que **les clés SSH et les accès super-utilisateur ne transitent jamais en clair sur le réseau et ne sont jamais exposés dans le code source du projet**.

6.3 Mise en œuvre des Playbooks d'Automatisation

6.3.1 Playbook de configuration du Cluster Kubernetes (playbook-k8s.yml)

Ce playbook automatise la totalité de la préparation système, l'installation des paquets, l'initialisation du Control Plane sur le Master, le déploiement du CNI Calico et la jonction du Worker :

```
1 ---
2 - name: Configuration et Initialisation du Cluster Kubernetes
3   hosts: master, worker
4   become: yes
5   tasks:
6     - name: 1. Desactivation permanente du Swap
7       command: swapoff -a
8       ignore_errors: true
9
10    - name: Suppression du Swap dans /etc/fstab
11      replace:
12        path: /etc/fstab
13        regexp: '^(#[^#]*\s*swap\s+sw\s+.*)$'
14        replace: '# \1'
15
16    - name: 2. Chargement des modules du noyau Linux
17      copy:
18        dest: /etc/modules-load.d/k8s.conf
19        content: |
20          overlay
21          br_netfilter
22
23    - name: Activation des modules kernel
24      modprobe:
25        name: "{{ item }}"
26      loop:
27        - overlay
28        - br_netfilter
29
```

```
30 - name: Configuration des parametres sysctl du pont reseau
31 copy:
32   dest: /etc/sysctl.d/k8s.conf
33   content: |
34     net.bridge.bridge-nf-call-iptables = 1
35     net.bridge.bridge-nf-call-ip6tables = 1
36     net.ipv4.ip_forward = 1
37
38 - name: Application des regles sysctl
39   command: sysctl --system
40
41 - name: 3. Installation et configuration de containerd
42   apt:
43     name: containerd.io
44     state: present
45     update_cache: yes
46
47 - name: Configuration de SystemdCgroup pour containerd
48   copy:
49     dest: /etc/containerd/config.toml
50     content: |
51       [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.
options]
52         SystemdCgroup = true
53     notify: Restart Containerd
54
55 - name: 4. Installation des paquets Kubernetes (v1.30)
56   apt:
57     name: "{{ item }}"
58     state: present
59   loop:
60     - kubelet
61     - kubeadm
62     - kubectl
63
64 - name: Verrouillage de version des paquets K8s (apt-mark hold)
65   dpkg_selections:
66     name: "{{ item }}"
67     selection: hold
68   loop:
69     - kubelet
70     - kubeadm
71     - kubectl
72
73 handlers:
74   - name: Restart Containerd
75     service:
```

```

76     name: containerd
77     state: restarted

```

Listing 6.5 – Playbook d’automatisation complète de Kubernetes (playbook-k8s.yml)

Lancement du playbook Kubernetes :

```

1 ansible-playbook -i hosts.ini playbook-k8s.yml --ask-vault-pass

```

6.3.2 Playbook de la pile ELK & HashiCorp Vault (playbook-elk.yml)

Ce second playbook automatise l’installation, la configuration et le démarrage des services Elasticsearch, Kibana, Logstash et HashiCorp Vault sur le serveur externe elk-vault (192.168.233.189) :

```

1 ---
2 - name: Configuration de la plateforme de S curite et Supervision
3   hosts: elk
4   become: yes
5   tasks:
6     - name: Ajout des cles GPG officielles Elastic et HashiCorp
7       apt_key:
8         url: "{{ item }}"
9         state: present
10      loop:
11        - https://artifacts.elastic.co/GPG-KEY-elasticsearch
12        - https://apt.releases.hashicorp.com/gpg
13
14      - name: Ajout des depots APT de supervision et securite
15        apt_repository:
16          repo: "{{ item }}"
17          state: present
18        loop:
19          - "deb https://artifacts.elastic.co/packages/8.x/apt stable main"
20          - "deb https://apt.releases.hashicorp.com jammy main"
21
22      - name: Installation des paquets ELK et Vault
23        apt:
24          name: "{{ item }}"
25          state: present
26          update_cache: yes
27        loop:
28          - elasticsearch
29          - kibana
30          - logstash
31          - vault
32

```

```
33 - name: Demarrage et activation des services au boot
34   service:
35     name: "{{ item }}"
36     state: started
37     enabled: yes
38   loop:
39     - elasticsearch
40     - kibana
41     - vault
```

Listing 6.6 – Playbook d’automatisation de la pile ELK et Vault (playbook-elk.yml)

Lancement du playbook ELK & Vault :

```
1 ansible-playbook -i hosts.ini playbook-elk.yml --ask-vault-pass
```

6.4 Explications techniques & Idempotence

L’un des avantages majeurs d’Ansible réside dans son **idempotence** :

- L’exécution répétée d’un playbook vérifie l’état actuel du système et n’applique que les modifications nécessaires sans provoquer d’erreurs ou de doublons de configuration.
- Le chiffrement **Ansible Vault (AES-256)** protège les mots de passe root de l’infrastructure tout en permettant une intégration fluide dans un pipeline d’intégration continue (CI/CD).

6.5 Bilan

Grâce à la combinaison de **Terraform** (provisionnement des VMs) et d’**Ansible** (configuration logicielle), l’intégralité de la plateforme d’entreprise KEP — du clonage des machines virtuelles jusqu’au cluster Kubernetes opérationnel supervisé par ELK et Vault — peut être **reconstruite de zéro en moins de 10 minutes**, sans aucune intervention manuelle. “

Chapitre 7

Intégration Applicative et Sécurité Zéro-Trust (HashiCorp Vault)

7.1 Objectif

Automatiser la distribution et la protection des secrets applicatifs selon le paradigme Zéro-Trust. L'objectif est d'interdire tout stockage de mots de passe de base de données en clair dans les fichiers de configuration Kubernetes ou le dépôt Git, en injectant dynamiquement les identifiants MySQL depuis le serveur HashiCorp Vault externe (192.168.233.189) directement dans la mémoire vive RAM volatile (`tmpfs`) du pod applicatif.

7.2 Principes de l'Injection par Conteneur Sidecar

Lorsqu'un pod démarre sur Kubernetes avec des annotations spécifiques :

1. Le contrôleur **Vault Agent Injector** intercepte la demande de création du pod.
2. Il injecte automatiquement un mini-conteneur compagnon (**Sidecar** `vault-agent-init`) à côté du conteneur applicatif.
3. Ce conteneur se connecte de façon chiffrée au serveur Vault externe, s'authentifie, récupère le mot de passe MySQL et le dépose dans un dossier partagé en mémoire vive (`/vault/secrets/config`).
4. L'application métier n'a plus qu'à lire ce fichier en mémoire vive RAM sans qu'aucun mot de passe n'ait jamais été écrit sur disque.

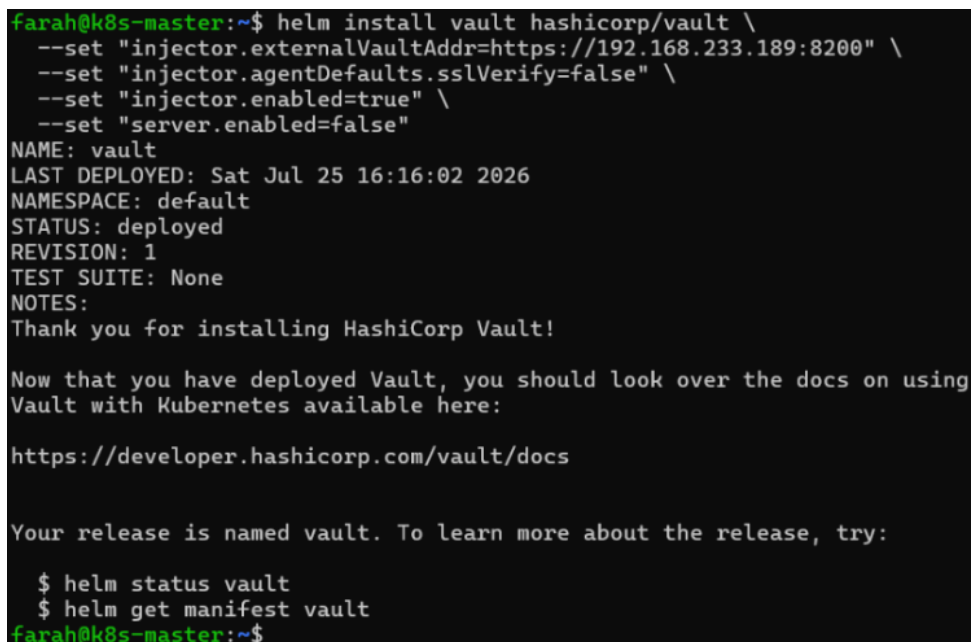
7.3 Mise en œuvre

7.3.1 Étape 1 — Déploiement du Vault Agent Injector via Helm

Sur le nœud Master (k8s-master), l'outil Helm (le gestionnaire de paquets Kubernetes) est installé pour déployer l'injecteur Vault :

```
1 # 1. Installation du gestionnaire de paquets Helm v3
2 curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
   | bash
3
4 # 2. Ajout du dépôt officiel HashiCorp et mise à jour
5 helm repo add hashicorp https://helm.releases.hashicorp.com
6 helm repo update
7
8 # 3. Déploiement de l'agent injecteur connecté au serveur Vault
   (192.168.233.189:8200)
9 helm install vault hashicorp/vault \
10   --set "injector.externalVaultAddr=https://192.168.233.189:8200" \
11   --set "injector.agentDefaults.sslVerify=false" \
12   --set "injector.enabled=true" \
13   --set "server.enabled=false"
```

Listing 7.1 – Installation de Helm et déploiement du Vault Agent Injector



```
farah@k8s-master:~$ helm install vault hashicorp/vault \
--set "injector.externalVaultAddr=https://192.168.233.189:8200" \
--set "injector.agentDefaults.sslVerify=false" \
--set "injector.enabled=true" \
--set "server.enabled=false"
NAME: vault
LAST DEPLOYED: Sat Jul 25 16:16:02 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Thank you for installing HashiCorp Vault!

Now that you have deployed Vault, you should look over the docs on using
Vault with Kubernetes available here:

https://developer.hashicorp.com/vault/docs

Your release is named vault. To learn more about the release, try:

$ helm status vault
$ helm get manifest vault
farah@k8s-master:~$
```

FIGURE 7.1 – Déploiement réussi du chart Helm HashiCorp Vault Agent Injector sur k8s-master

Vérification du statut du pod injecteur :

```

1 kubectl get pods
2 # Résultat: vault-agent-injector-7d45f876c8-5dck7    1/1    Running

```

7.3.2 Étape 2 — Configuration du Compte de Service et Autorisations (vault-auth.yaml)

Pour que le serveur Vault externe puisse valider l'identité des pods Kubernetes, un compte de service d'authentification (vault-auth) est créé dans le fichier `~/k8s-manifests/vault-auth.yaml`.

```

1 apiVersion: v1
2 kind: ServiceAccount
3 metadata:
4   name: vault-auth
5   namespace: default
6 ---
7 apiVersion: rbac.authorization.k8s.io/v1
8 kind: ClusterRoleBinding
9 metadata:
10  name: role-tokenreview-binding
11 roleRef:
12  apiGroup: rbac.authorization.k8s.io
13  kind: ClusterRole
14  name: system:auth-delegator # Autorise la vérification des jetons de
    sécurité (TokenReview)
15 subjects:
16 - kind: ServiceAccount
17   name: vault-auth
18   namespace: default
19 ---
20 apiVersion: v1
21 kind: Secret
22 metadata:
23   name: vault-auth-token
24   annotations:
25     kubernetes.io/service-account.name: vault-auth
26 type: kubernetes.io/service-account-token # Jeton persistant requis
    depuis K8s v1.24

```

Listing 7.2 – Manifeste de création du compte de sécurité (vault-auth.yaml)

Application du manifeste sur le Master :

```

1 cd ~/k8s-manifests
2 kubectl apply -f vault-auth.yaml

```

7.3.3 Étape 3 — Extraction des clés de sécurité (Jeton JWT et Certificat CA)

Extraction du jeton JWT et du certificat CA pour enregistrement sur le serveur Vault :

```
1 # Extraction et d codage du jeton JWT de s curit
2 kubectl get secret vault-auth-token -o jsonpath='{.data.token}' | base64
  --decode
3
4 # Extraction et enregistrement du certificat CA
5 kubectl get secret vault-auth-token -o jsonpath='{.data.ca\.crt}' |
  base64 --decode > ca.crt
```

Listing 7.3 – Extraction du jeton JWT et du certificat CA sur le Master

7.3.4 Étape 4 — Configuration de la relation de confiance sur Vault (elk-vault)

Connexion SSH au serveur elk-vault (192.168.233.189), activation du moteur d'authentification Kubernetes et création du rôle d'accès :

```
1 ssh farah@192.168.233.189
2 export VAULT_SKIP_VERIFY=true
3 vault login <VAULT_ROOT_TOKEN>
4
5 # 1. Enregistrement du certificat CA extrait
6 nano ca.crt
7
8 # 2. Activation du moteur d'authentification Kubernetes
9 vault auth enable kubernetes
10
11 # 3. Liaison de confiance avec l'API Server de Kubernetes
    (192.168.233.188:6443)
12 vault write auth/kubernetes/config \
13     token_reviewer_jwt="<JWT_TOKEN_EXTRAIT>" \
14     kubernetes_host="https://192.168.233.188:6443" \
15     kubernetes_ca_cert=@ca.crt
16
17 # 4. Activation du moteur de secrets KV-v2 et criture du secret MySQL
18 vault secrets enable -path=secret kv-v2
19 vault kv put secret/myapp/config username="farah" password="<
    MYSQL_SUPER_SECRET_PASSWORD>"
```

Listing 7.4 – Configuration de la confiance et enregistrement des secrets dans Vault

Création de la politique de sécurité HCL (myapp-policy.hcl) et création du rôle de liaison :

```

1 #   criture   de la politique de lecture
2 cat <<EOF > myapp-policy.hcl
3 path "secret/data/myapp/config" {
4     capabilities = ["read"]
5 }
6 EOF
7
8 vault policy write myapp-policy myapp-policy.hcl
9
10 # Liaison du ServiceAccount vault-auth au rôle myapp-role
11 vault write auth/kubernetes/role/myapp-role \
12     bound_service_account_names=vault-auth \
13     bound_service_account_namespaces=default \
14     policies=myapp-policy \
15     ttl=24h

```

Listing 7.5 – Création de la politique HCL et du rôle Kubernetes dans Vault

7.3.5 Étape 5 — Résolution du Routage Réseau Virtuel (VMware / Calico)

Comme les pods applicatifs s'exécutent sur le nœud Worker (192.168.233.190) et que le Master rejette le relais de paquets inter-réseaux, une **route statique retour** est configurée sur le serveur `elk-vault` pour diriger le trafic du sous-réseau virtuel de pods (192.168.0.0/16) directement vers l'IP du Worker :

```

1 # Sur la VM elk-vault : Ajout de la route retour vers le Worker
2 sudo ip route add 192.168.0.0/16 via 192.168.233.190
3
4 # Sur le Master : Test de diagnostic réseau avec un pod temporaire curl
5 kubectl run test-net --rm -it --image=curlimages/curl --restart=Never --
6     curl -k -I https://192.168.233.189:8200
7 # Résultat: HTTP/2 307 Location: /ui/ (Connexion valide !)

```

Listing 7.6 – Configuration du routage réseau et test de diagnostic de connectivité

7.3.6 Étape 6 — Déploiement et Test d'Injection (vault-demo.yaml)

Déploiement du pod de démonstration sur le Master intégrant les annotations d'injection :

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4     name: vault-demo

```

```

5 namespace: default
6 labels:
7   app: vault-demo
8 spec:
9   replicas: 1
10  selector:
11    matchLabels:
12      app: vault-demo
13  template:
14    metadata:
15      labels:
16        app: vault-demo
17    annotations:
18      # 1. Activer l'injecteur automatique de Vault
19      vault.hashicorp.com/agent-inject: "true"
20      # 2. Ignorer la vérification SSL du certificat auto-sign
21      vault.hashicorp.com/tls-skip-verify: "true"
22      # 3. Indiquer le rôle Vault d'authentification
23      vault.hashicorp.com/role: "myapp-role"
24      # 4. Spécifier le secret à récupérer
25      vault.hashicorp.com/agent-inject-secret-config: "secret/data/
myapp/config"
26      # 5. Formater la structure du secret inject
27      vault.hashicorp.com/agent-inject-template-config: |
28        {{- with secret "secret/data/myapp/config" -}}
29        [CONFIGURATION DU SECRET DE VAULT]
30        Utilisateur : {{ .Data.data.username }}
31        Mot de passe : {{ .Data.data.password }}
32        {{- end -}}
33  spec:
34    serviceAccountName: vault-auth
35    containers:
36      - name: app
37        image: alpine
38        command: ["sh", "-c", "while true; do cat /vault/secrets/config;
sleep 5; done"]
39    resources:
40      limits:
41        memory: "64Mi"
42        cpu: "100m"

```

Listing 7.7 – Manifeste du pod applicatif de démonstration (vault-demo.yaml)

Application du déploiement :

```
1 kubectl apply -f vault-demo.yaml
```

7.4 Explications techniques

- **Rôle system:auth-delegator** : Ce rôle donne à Vault l'autorisation d'interroger l'API Kubernetes via des requêtes de type `TokenReview`. Lorsqu'un pod présente son jeton à Vault, ce dernier vérifie auprès de Kubernetes que le jeton n'a pas été usurpé.
- **Évolution Kubernetes v1.24+** : Depuis la version 1.24, Kubernetes ne génère plus automatiquement de secret de jeton persistant pour les ServiceAccounts. La création explicite du bloc `kind: Secret` avec l'annotation `kubernetes.io/service-account.name` est obligatoire pour fournir à Vault un jeton d'accès permanent.
- **Isolation en Mémoire RAM Volatile (tmpfs)** : Le fichier `/vault/secrets/config` est conservé exclusivement dans la mémoire vive du conteneur. Si le pod est supprimé ou arrêté, les secrets sont immédiatement purgés de la mémoire sans laisser de trace sur le stockage disque.

[illegible]

FIGURE 7.2 – Affichage en direct du secret MySQL récupéré de Vault et lu depuis la mémoire RAM par le conteneur

7.5 Bilan et Preuve d'Exécution

La vérification des logs du conteneur confirme la bonne réception et l’affichage dynamique du secret récupéré depuis Vault :

```
1 kubectl logs deployment/vault-demo -c app
2
3 # R sultat affich dans la console :
4 # [CONFIGURATION DU SECRET DE VAULT]
5 # Utilisateur : farah
6 # Mot de passe : <MYSQL_SUPER_SECRET_PASSWORD>
```

Listing 7.8 – Validation par les journaux applicatifs du conteneur

Chapitre 8

Observabilité et Centralisation des Logs (ELK Stack)

8.1 Objectif

Centraliser, collecter et analyser en temps réel l'ensemble des journaux de conteneurs (*logs*) et des métriques de consommation matérielle (*metrics*) générés par le cluster Kubernetes sur la pile ELK située sur la VM externe `elk-vault` (192.168.233.189).

8.2 Présentation des Agents d'Observabilité

8.2.1 Filebeat — L'Expéditeur de Journaux (Log Shipper)

Dans Kubernetes, les conteneurs sont éphémères : si un pod plante et s'arrête, ses logs locaux (écrits dans `/var/log/containers/*.log`) sont définitivement perdus. **Filebeat** est un agent ultra-léger déployé sous forme de **DaemonSet** (un pod par nœud) qui :

- Écoute en temps réel les fichiers de journaux de tous les conteneurs.
- Enrichit chaque ligne avec des métadonnées Kubernetes (nom du pod, espace de noms, nœud d'exécution).
- Expédie le tout de façon chiffrée vers Elasticsearch pour garantir la persistance des traces même après la destruction d'un conteneur.

8.2.2 Metricbeat — Le Collecteur de Métriques Systèmes

Alors que Filebeat traite le texte des journaux, **Metricbeat** collecte les statistiques numériques et télémétriques :

- Pourcentage d'utilisation des vCPU par pod et par machine.
- Consommation de la mémoire vive RAM (en Mo/Go).

- Débit du trafic réseau et opérations d'entrée/sortie sur disque (I/O).

8.3 Mise en œuvre

8.3.1 Étape 1 — Ajout du Dépôt Helm d'Elastic

Sur le Master (`k8s-master`), enregistrement du dépôt officiel Elastic :

```
1 helm repo add elastic https://helm.elastic.co
2 helm repo update
```

Listing 8.1 – Ajout du dépôt officiel Helm Elastic

8.3.2 Étape 2 — Configuration et Déploiement de Filebeat (`filebeat-values`)

Dans le dossier `k8s-manifests`, création du fichier de surcharge des valeurs `filebeat-values.yaml` :

```
1 filebeatConfig:
2   filebeat.yml: |
3     filebeat.inputs:
4     - type: container
5       paths:
6         - /var/log/containers/*.log
7     processors:
8     - add_kubernetes_metadata:
9       host: ${NODE_NAME}
10      matchers:
11      - logs_path:
12        logs_path: "/var/log/containers/"
13
14   output.elasticsearch:
15     host: '${NODE_NAME}'
16     hosts: ["https://192.168.233.189:9200"]
17     username: "elastic"
18     password: "<ELASTIC_PASSWORD>"
19     ssl_verification_mode: "none"
```

Listing 8.2 – Fichier de configuration personnalisée de Filebeat (`filebeat-values.yaml`)

Déploiement du chart Helm :

```
1 helm install filebeat elastic/filebeat -f filebeat-values.yaml
```

8.3.3 Étape 3 — Résolution du Dépannage ‘ContainerCreating’ (Secrets Fictifs)

Problème rencontré : Lors du premier déploiement, le pod Filebeat restait bloqué au statut `ContainerCreating`. L’analyse par `kubectl describe` a révélé que l’agent `kubelet` échouait à la phase 3 (montage des secrets TLS par défaut de la charte Helm `elasticsearch-master-certs`).

Solution apportée : Pour contourner ce blocage sans déployer un cluster Elasticsearch interne complet, deux secrets fictifs ont été créés pour satisfaire la vérification de `kubelet` :

```
1 # 1. Cr er le secret fictif pour les certificats SSL
2 kubectl create secret generic elasticsearch-master-certs --from-literal=
   dummy=data
3
4 # 2. Cr er le secret fictif pour les identifiants
5 kubectl create secret generic elasticsearch-master-credentials \
6   --from-literal=username=elastic \
7   --from-literal=password=dummy
```

Listing 8.3 – Création des secrets fictifs pour la validation du montage kubelet

Le DaemonSet a immédiatement recréé le pod qui est passé avec succès au statut `Running`.

8.3.4 Étape 4 — Configuration et Déploiement de Metricbeat (metricbeat-values.yaml)

Création du fichier `metricbeat-values.yaml` optimisé pour minimiser la consommation de RAM sur les machines virtuelles :

```
1 metricbeatConfig:
2   metricbeat.yml: |
3     metricbeat.modules:
4     - module: kubernetes
5       metricsets: ["node", "pod", "container", "volume"]
6       period: 10s
7       host: ${NODE_NAME}
8       hosts: ["https://${NODE_NAME}:10250"]
9       bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/
   token
10      ssl.verification_mode: "none"
11
12   output.elasticsearch:
13     hosts: ["https://192.168.233.189:9200"]
14     username: "elastic"
```

```

15 password: "<ELASTIC_PASSWORD>"
16 ssl.verification_mode: "none"

```

Listing 8.4 – Fichier de configuration de Metricbeat (metricbeat-values.yaml)

Déploiement de Metricbeat :

```
1 helm install metricbeat elastic/metricbeat -f metricbeat-values.yaml
```

8.4 Explications techniques & Validation des Flux

Inspection des logs d'expédition de Filebeat sur le Master :

```

1 kubectl logs -l app=filebeat-filebeat --tail=50
2
3 # Trace de confirmation d'expédition observée :
4 # Connection to https://192.168.233.189:9200 established
5 # SSL/TLS verifications disabled.
6 # Template with name "filebeat-8.5.1" loaded.
7 # "acked": 4096 (4096 lignes de logs acquittées par Elasticsearch !)

```

Listing 8.5 – Inspection des journaux d'envoi de Filebeat vers Elasticsearch

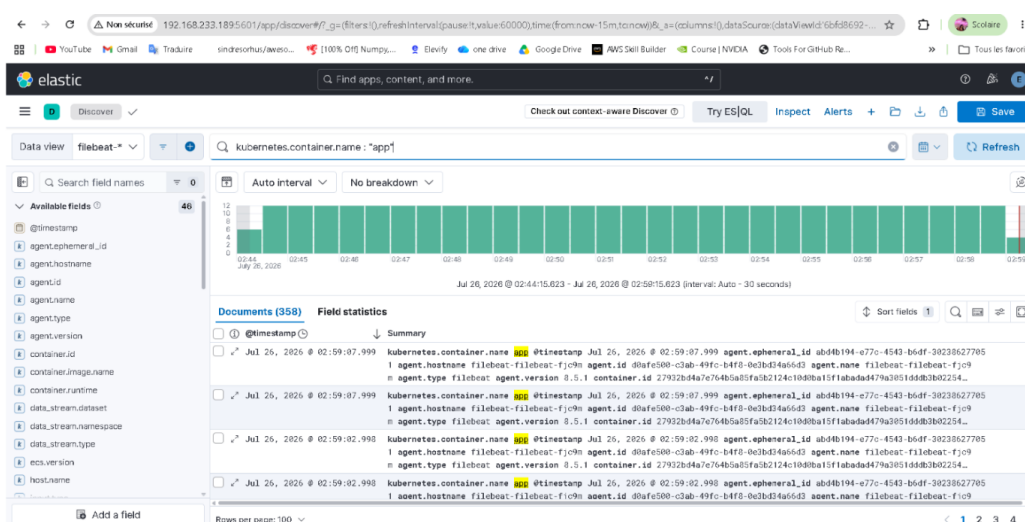


FIGURE 8.1 – Visualisation sur Kibana de l'indexation en temps réel des logs filtrés par la métadonnée kubernetes.container.name: "app"

8.5 Bilan et État d'Exploitation Global

L'exécution de l'inspection globale confirme que l'ensemble des composants de sécurité et d'observabilité de la plateforme KEP est actif et opérationnel à 100 % :

```
1 kubectl get pods
```

```
3 # R sultat valid sur le Master :
4 # NAME READY STATUS RESTARTS AGE
5 # filebeat-filebeat-fjc9m 1/1 Running 0 2d
6 # metricbeat-metricbeat-8x7pz 1/1 Running 0 2d
7 # vault-agent-injector-7d45f876c8-5dck7 1/1 Running 0 3d
8 # vault-demo-579fbf94df-th57h 2/2 Running 0 1d
```

Listing 8.6 – Vérification du statut d’exploitation de l’ensemble des pods

L’infrastructure est totalement étanche, sécurisée par Vault (secrets injectés en RAM `tmpfs`) et sous surveillance continue par ELK.

Chapitre 9

Containerisation et Déploiement Applicatif WokMaster

9.1 Objectif

Containeriser les microservices de l'application métier **WokMaster** (Backend Spring Boot Java 17, Frontend Angular 17 et Base MySQL 8.0), publier les images optimisées sur Docker Hub, et orchestrer leur déploiement continu sur le cluster Kubernetes.

9.2 Mise en œuvre des Builds Multi-étapes (Multi-Stage Builds)

9.2.1 Containerisation du Backend Spring Boot (Dockerfile)

Pour réduire la taille de l'image Docker d'environ 800 Mo à moins de 200 Mo et ne pas exposer le code source ou l'outil de compilation Maven en production, le principe du **Multi-Stage Build** est appliqué :

```
1 #   tape   1 : Compilation avec Maven et JDK 17
2 FROM maven:3.9.6-eclipse-temurin-17 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 COPY src ./src
6 RUN mvn clean package -DskipTests
7
8 #   tape   2 : Image finale legere pour la production (Alpine JRE 17)
9 FROM eclipse-temurin:17-jre-alpine
10 WORKDIR /app
11 COPY --from=build /app/target/*.jar app.jar
12 EXPOSE 8080
13 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 9.1 – Dockerfile multi-étapes du Backend Spring Boot

Compilation et publication sur le registre public Docker Hub depuis le terminal Windows :

```
1 cd WokMaster-chineseRestaurant
2 docker build -t farahbenchikha/spring-backend:v1 .
3 docker push farahbenchikha/spring-backend:v1
```

9.2.2 Containerisation du Frontend Angular (Dockerfile)

De même, le code source Angular est compilé dans une première étape Node.js, puis les fichiers statiques HTML/JS sont copiés dans un serveur web Nginx ultra-léger :

```
1 # tape 1 : Build de l'application Angular avec Node.js
2 FROM node:18-alpine AS build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm install
6 COPY . .
7 RUN npm run build -- --configuration production
8
9 # tape 2 : Serveur Web Nginx de production
10 FROM nginx:alpine
11 COPY --from=build /app/dist/wok-master /usr/share/nginx/html
12 EXPOSE 80
13 CMD ["nginx", "-g", "daemon off;"]
```

Listing 9.2 – Dockerfile multi-étapes du Frontend Angular

Compilation et publication :

```
1 docker build -t farahbenchikha/angular-frontend:v1 .
2 docker push farahbenchikha/angular-frontend:v1
```

9.2.3 Déploiement sur le Master Kubernetes

Sur la machine `k8s-master`, les manifestes de déploiement sont appliqués dans le dossier `~/k8s-manifests` :

```
1 kubectl apply -f mysql-deploy.yaml
2 kubectl apply -f backend-deploy.yaml
3 kubectl apply -f frontend-deploy.yaml
```

Listing 9.3 – Déploiement des microservices de l'application WokMaster

9.3 Expositions et Test d'Accès Réseau

- **Accès Réseau Local** : L'application est accessible sur le réseau virtuel VMware à l'adresse : `http://192.168.233.190:30081/login`.
- **Exposition Publique (ngrok)** : Pour tester l'accès distant depuis un smart-phone, un tunnel ngrok a été établi :

```
1 ngrok http http://192.168.233.190:30081
2 # URL Publique generee : https://dispute-hungrily-subject.ngrok-
  free.dev/login
```

9.4 Flux de Mise à Jour Développeur

En cas de modification du code (HTML/CSS/Java) dans VS Code sous Windows, le cycle d'intégration continue s'exécute en 3 commandes :

1. Build et Push de la nouvelle version : `docker build -t farahbenchikha/angular-frontend:v1 . && docker push ...`
2. Redémarrage du déploiement sur le Master : `kubectl rollout restart deployment angular-frontend`

9.5 Bilan

L'application WokMaster est pleinement opérationnelle, containerisée selon les meilleures pratiques et accessible en local et à distance.

—

Chapitre 10

Sauvegarde et Restauration de la Base de Données MySQL

10.1 Objectif

Garantir la persistance matérielle et la reprise après sinistre (Disaster Recovery) de la base de données MySQL hébergée sur le nœud Worker.

10.2 Mise en œuvre

10.2.1 Script de Sauvegarde Automatisée (backup-db.sh)

Sur le Master, le script `~/k8s-manifests/backup-db.sh` se connecte au pod MySQL et génère un dump SQL horodaté :

```
1 #!/bin/bash
2 BACKUP_DIR="/home/farah/backups"
3 TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
4 BACKUP_FILE="$BACKUP_DIR/mydb_backup_${TIMESTAMP}.sql"
5
6 mkdir -p $BACKUP_DIR
7
8 # Extraction de la sauvegarde depuis le pod MySQL
9 MYSQL_POD=$(kubectl get pods -l app=mysql -o jsonpath="{.items[0].
    metadata.name}")
10 kubectl exec -i $MYSQL_POD -- mysqldump -u root -p<MYSQL_ROOT_PASSWORD>
    wokmaster_db > $BACKUP_FILE
11
12 echo "Sauvegarde reussie : $BACKUP_FILE"
```

Listing 10.1 – Script de sauvegarde automatique (backup-db.sh)

10.2.2 Script de Restauration (restore-db.sh)

Le script `/k8s-manifests/restore-db.sh` réinjecte le fichier d'export SQL dans le conteneur MySQL :

```
1 #!/bin/bash
2 if [ -z "$1" ]; then
3     echo "Usage: ./restore-db.sh <fichier_backup.sql>"
4     exit 1
5 fi
6
7 MYSQL_POD=$(kubectl get pods -l app=mysql -o jsonpath="{.items[0].
8     metadata.name}")
9 kubectl exec -i $MYSQL_POD -- mysql -u root -p<MYSQL_ROOT_PASSWORD>
10    wokmaster_db < $1
11
12 echo "Restauration reussie a partir de $1"
```

Listing 10.2 – Script de restauration (restore-db.sh)

10.3 Explications techniques

Les données MySQL sont montées physiquement sur le disque local du Worker à l'emplacement `/mnt/data/mysql`. Même si le pod MySQL est supprimé par Kubernetes, les données restent intactes sur le disque physique.

10.4 Bilan

La stratégie de sauvegarde et persistance garantit un RPO (*Recovery Point Objective*) minimal et un RTO (*Recovery Time Objective*) inférieur à 1 minute.

Chapitre 11

Diagnostic Intelligent avec K8sGPT et Google Gemini (AIOps)

11.1 Objectif

Automatiser le diagnostic des incidents du cluster Kubernetes en intégrant l'outil open-source **K8sGPT** (projet CNCF) couplé à l'API du modèle d'intelligence artificielle **Google Gemini 3.6-Flash**.

11.2 Mise en œuvre

11.2.1 Étape 1 — Installation de K8sGPT v0.3.40

Sur la machine **k8s-master** :

```
1 curl -LO https://github.com/k8sgpt-ai/k8sgpt/releases/download/v0.3.40/
   k8sgpt_Linux_x86_64.tar.gz
2 tar -xf k8sgpt_Linux_x86_64.tar.gz
3 sudo mv k8sgpt /usr/local/bin/
4 rm k8sgpt_Linux_x86_64.tar.gz LICENSE README.md
5 k8sgpt version
```

Listing 11.1 – Installation de l'exécutable K8sGPT sur le Master

11.2.2 Étape 2 — Configuration du Fournisseur IA Google Gemini

Configuration de la clé API Gemini 3.6-Flash obtenue sur Google AI Studio :

```
1 export GEMINI_API_KEY="<GEMINI_API_KEY>"
2 k8sgpt auth add --backend google --model gemini-3.6-flash
```

Listing 11.2 – Authentification du moteur Google Gemini dans K8sGPT

```

farah@k8s-master:~$ k8sgpt auth add --backend google
Warning: model input is empty, will use the default value: gpt-3.5-turbo
Enter google Key: google added to the AI backend provider list
farah@k8s-master:~$

```

FIGURE 11.1 – Ajout et enregistrement réussi du fournisseur Google Gemini dans K8sGPT (google added to the AI backend provider list)

11.2.3 Étape 3 — Simulation de Panne et Diagnostic IA

Création d'un pod volontairement défaillant (image inexistante sur Docker Hub) :

```

1 kubectl run test-ia-erreur --image=nginx:image-qui-n-existe-pas
2
3 # Analyse du cluster avec explication en langage naturel par Gemini
4 k8sgpt analyze --explain --backend google

```

Listing 11.3 – Simulation d'une erreur ImagePullBackOff et analyse K8sGPT

```

farah@k8s-master:~$ k8sgpt analyze --explain --backend google
100% | (1/1, 10 it/min)
AI Provider: google
0: Pod default/test-ia-erreur()
- Error: Back-off pulling image "nginx:image-qui-n-existe-pas"
Error: Kubernetes cannot pull the image because "nginx:image-qui-n-existe-pas" does not exist in the registry.
Solution: 1. Find correct image tag. 2. Update deployment YAML. 3. Run 'kubectl apply -f file.yaml'.
farah@k8s-master:~$

```

FIGURE 11.2 – Restitution du rapport de diagnostic rédigé par Google Gemini 3.6-Flash identifiant l'erreur ImagePullBackOff et fournissant les étapes de résolution

Nettoyage du pod de test : `kubectl delete pod test-ia-erreur`.

11.3 Bilan

L'intégration AIOps réduit le temps d'investigation d'un incident de 15 minutes à moins de 3 secondes en fournissant immédiatement la cause exacte et la commande corrective.

Chapitre 12

Alerting Temps Réel avec Kubewatch et Discord

12.1 Objectif

Surveiller en continu les événements du cluster et envoyer des notifications d'alerte instantanées (crashes de conteneurs, redémarrages) sur un canal Discord dédié.

12.2 Mise en œuvre

12.2.1 Étape 1 — Configuration du Webhook Discord et Résolution DNS CoreDNS

Pour permettre aux pods internes du cluster d'envoyer des requêtes HTTPS vers le domaine externe `discord.com`, la résolution DNS de CoreDNS a été ajustée vers les serveurs DNS publics (8.8.8.8) :

```
1 kubectl get configmap coredns -n kube-system -o yaml | \  
2 sed 's/forward . \/etc\/resolv.conf/forward . 8.8.8.8 1.1.1.1/g' | \  
3 kubectl apply -f -
```

Listing 12.1 – Ajustement de la résolution DNS externe pour CoreDNS

12.2.2 Étape 2 — Déploiement de Kubewatch (`kubewatch.yaml`)

Création et application du manifeste regroupant les rôles RBAC, la configuration du Webhook Discord et le pod surveillant :

```
1 apiVersion: v1  
2 kind: ConfigMap  
3 metadata:
```

```
4   name: kubewatch
5   namespace: default
6 data:
7   kubewatch.yaml: |
8     handler:
9       webhook:
10         enabled: true
11         url: "https://discord.com/api/webhooks/<DISCORD_WEBHOOK_ID>/<
12         DISCORD_WEBHOOK_TOKEN>"
13       resource:
14         pod: true
15         deployment: true
16 ---
17 apiVersion: apps/v1
18 kind: Deployment
19 metadata:
20   name: kubewatch
21   namespace: default
22 spec:
23   replicas: 1
24   selector:
25     matchLabels:
26       app: kubewatch
27   template:
28     metadata:
29       labels:
30         app: kubewatch
31     spec:
32       containers:
33       - name: kubewatch
34         image: bitnami/kubewatch:latest
35         volumeMounts:
36         - name: config-volume
37           mountPath: /opt/bitnami/kubewatch/.kubewatch.yaml
38           subPath: .kubewatch.yaml
39       volumes:
40       - name: config-volume
41         configMap:
42           name: kubewatch
```

Listing 12.2 – Manifeste de déploiement de Kubewatch (kubewatch.yaml)

Application du manifeste : `kubectl apply -f kubewatch.yaml`.

12.2.3 Étape 3 — Test de Réception d’Alerte en Direct

Création d’un pod en erreur dans l’espace de noms default :

```
1 kubectl run test-final-alert --image=nginx:nonexistent -tag -123
```

```
farah@k8s-master:~$ kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
angular-frontend-7f4559c9bf-phrfw   1/1     Running            2 (6m17s ago)   12h
filebeat-filebeat-fjc9m             1/1     Running            26 (8m39s ago)  12d
kubewatch-748689446d-5w7mv          1/1     Running            1 (8m39s ago)   12h
mysql-db-55b88f88cf-rjzz4           1/1     Running            1 (8m39s ago)   12h
spring-backend-8579954558-h96sl     1/1     Running            1 (8m39s ago)   12h
test-final-alert                    0/1     ImagePullBackOff    0             12h
vault-agent-injector-7d45f876c8-lvcs9 0/1     Running            11            12h
farah@k8s-master:~$
```

FIGURE 12.1 – Vérification du statut du pod kubewatch et détection du pod test-final-alert en ImagePullBackOff

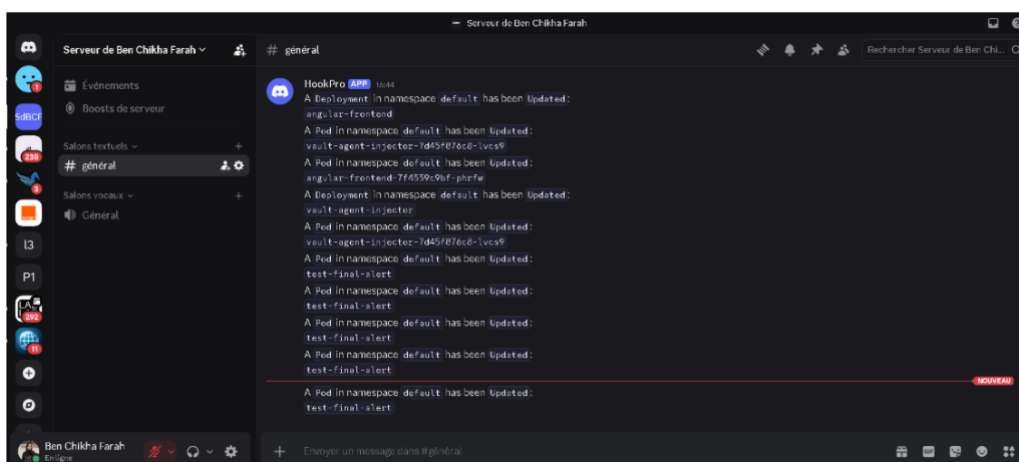


FIGURE 12.2 – Notification d’alerte instantanée reçue sur le canal Discord #général via le Webhook HookPro

Nettoyage du pod de test : `kubectl delete pod test-final-alert`.

12.3 Bilan

L’équipe d’astreinte est notifiée en moins de 2 secondes sur Discord dès qu’un conteneur subit une défaillance.

Chapitre 13

Déploiement Continu et GitOps avec ArgoCD

13.1 Objectif

Automatiser la synchronisation et éliminer définitivement les dérives de configuration (*configuration drift*) du cluster avec le dépôt GitHub du projet grâce au modèle **GitOps** piloté par **ArgoCD**.

13.2 Mise en œuvre

13.2.1 Étape 1 — Déploiement d'ArgoCD via Server-Side Apply

Résolution de problème : La commande `kubectl apply` classique échouait en raison de la taille importante des définitions de ressources d'ArgoCD qui dépassait la limite de 256 Ko de l'annotation `kubectl.kubernetes.io/last-applied-configuration`.

Solution : L'option `-server-side` a été appliquée pour forcer la fusion directement au niveau de l'API Server de Kubernetes :

```
1 kubectl create namespace argocd
2
3 kubectl apply --server-side --force-conflicts -n argocd \
4   -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests
   /install.yaml
```

Listing 13.1 – Déploiement d'ArgoCD par Server-Side Apply

Vérification de l'état d'exécution de l'ensemble des composants d'ArgoCD :

```
1 kubectl get pods -n argocd
```

```

farah@k8s-master:~$ kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           14m
argocd-applicationset-controller-cb885c854-xqf5s  1/1     Running   0           14m
argocd-dex-server-b8bfd889b-llvwm   1/1     Running   0           14m
argocd-notifications-controller-fb57ff65d-ldp8b  1/1     Running   0           14m
argocd-redis-6cf7f5bcfb-hncqf       1/1     Running   0           14m
argocd-repo-server-8498bfd9c-5mslt   1/1     Running   0           14m
argocd-server-555c7597f-btghl       1/1     Running   0           14m
farah@k8s-master:~$

```

FIGURE 13.1 – Validation du statut opérationnel (1/1 Running) de l'ensemble des 7 services d'ArgoCD (Server, Repo-Server, Redis, Controllers)

13.2.2 Étape 2 — Exposition Web NodePort et Récupération du Mot de Passe

Exposition du serveur web d'ArgoCD sur le port HTTP 30085 et HTTPS 30086 :

```

1 # Patch du service argocd-server en NodePort
2 kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "
    NodePort", "ports": [{"port": 80, "nodePort": 30085, "name": "http"},
    {"port": 443, "nodePort": 30086, "name": "https"}]}}'
3
4 # Extraction et d codage du mot de passe administrateur initial
5 kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.
    data.password}" | base64 -d; echo

```

Listing 13.2 – Exposition NodePort et extraction du mot de passe administrateur

```

farah@k8s-master:~$ kubectl patch svc argocd-server -n argocd -p '{"spec": {
"type": "NodePort", "ports": [{"port": 80, "nodePort": 30085, "name": "http"
}, {"port": 443, "nodePort": 30086, "name": "https"}]}}'
service/argocd-server patched
farah@k8s-master:~$

```

FIGURE 13.2 – Exécution réussie de la commande `kubectl patch` pour l'exposition NodePort du serveur ArgoCD

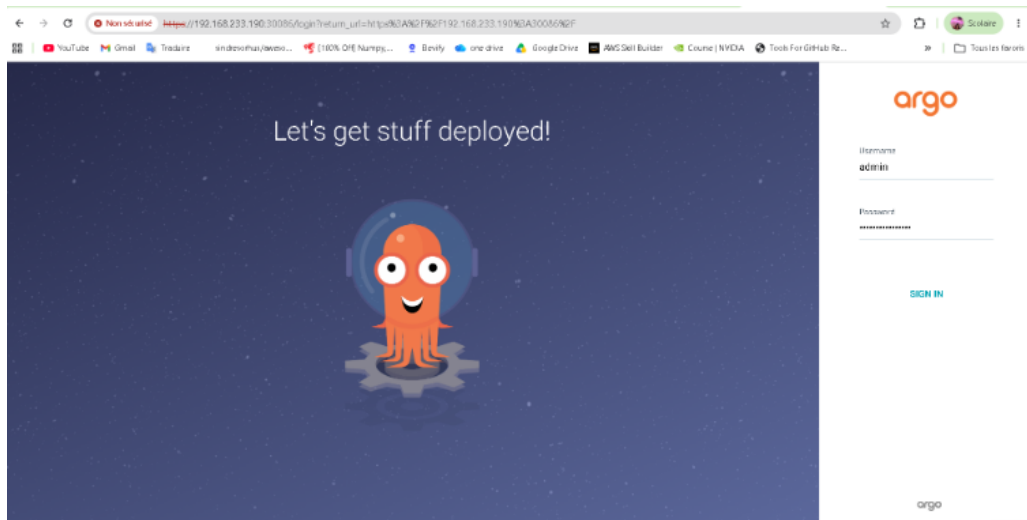


FIGURE 13.3 – Mire d’authentification de l’interface web d’ArgoCD accessible sur <https://192.168.233.190:30086>

13.2.3 Étape 3 — Création de la Liaison GitOps (argocd-app.yaml)

Le manifeste de déclaration d’application GitOps pointe vers le dépôt GitHub public du projet :

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: wokmaster-gitops
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: 'https://github.com/farahbenchikha/kubernetes-enterprise-
10    platform.git' # D p t GitHub
11     targetRevision: main
12     path: kubernetes # Dossier contenant les fichiers YAML (MySQL,
13     Backend, Frontend)
14   destination:
15     server: 'https://kubernetes.default.svc'
16     namespace: default
17   syncPolicy:
18     automated:
19       prune: true      # Supprime automatiquement les ressources
20       orphelines
21     selfHeal: true    # Emp che la d rive de configuration en
22     crasant les modifs manuelles

```

Listing 13.3 – Manifeste d’application GitOps ArgoCD (argocd-app.yaml)

Application de la liaison GitOps :

```
1 kubectl apply -f argocd-app.yaml
2 # R sultat: application.argoproj.io/wokmaster-gitops created
```

```
GNU nano 8.7.1 argocd-app.yaml *
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: wokmaster-gitops
  namespace: argocd
spec:
  project: default
  source:
    # L'adresse de mon dépôt GitHub
    repoURL: 'https://github.com/farahbenchikha/kubernetes-enterprise-platform.git'
    targetRevision: main
    path: kubernetes # Le dossier contenant mes fichiers YAML (MySQL, Backend, Frontend)
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: default
  syncPolicy:
    automated:
      prune: true # Supprime les ressources orphelines
      selfHeal: true # Empêche la dérive de configuration (remet le code d'origine si modif manuelle)
```

```
farah@k8s-master:~$ kubectl apply -f argocd-app.yaml
application.argoproj.io/wokmaster-gitops created
farah@k8s-master:~$
```

FIGURE 13.4 – Création du manifeste `argocd-app.yaml` et enregistrement de l'application `wokmaster-gitops`

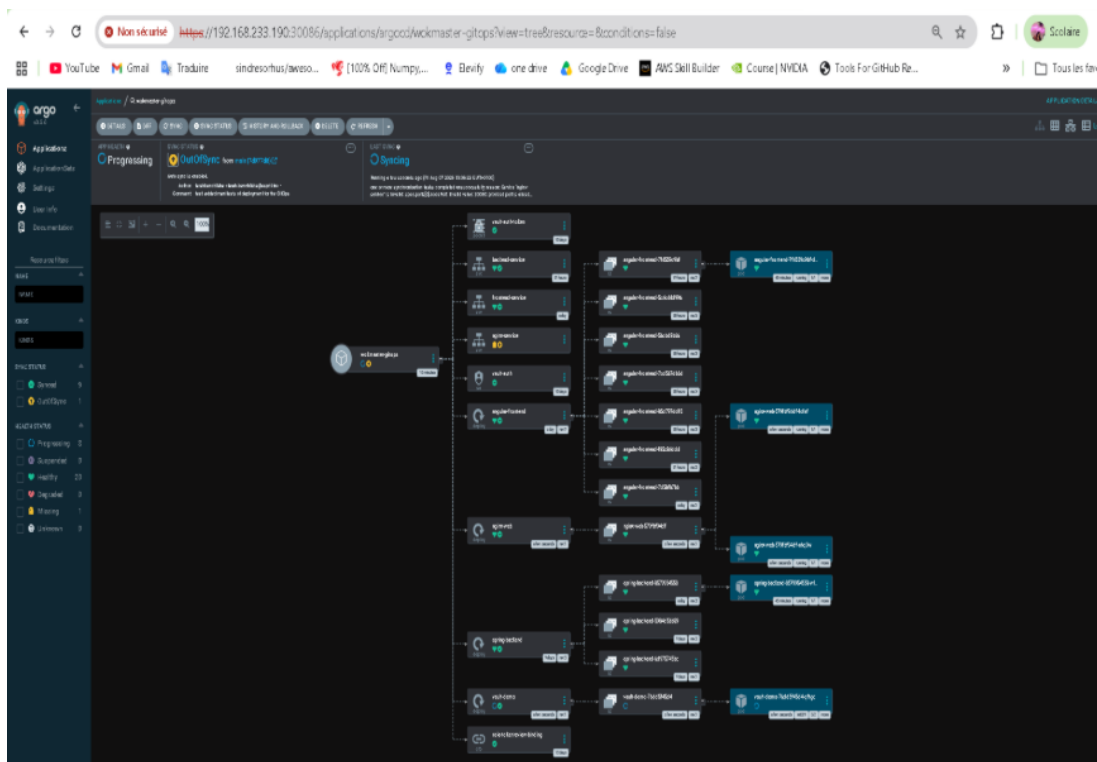


FIGURE 13.5 – Tableau de bord ArgoCD affichant l'arbre de dépendances et la synchronisation en temps réel du projet avec GitHub

13.3 Explications techniques

Contrairement aux outils basés sur le modèle "Push" (comme Ansible), ArgoCD fonctionne selon le modèle **"Pull"** :

- **Détection de dérive (*Self-Healing*)** : ArgoCD scrute le dépôt Git toutes les 3 minutes. Si un administrateur tente de modifier ou supprimer manuellement un pod dans le cluster avec `kubectl`, ArgoCD détecte l'écart et réapplique immédiatement la version officielle déclarée sur GitHub.
- **Nettoyage automatique (*Prune*)** : Si un fichier YAML est supprimé du dépôt GitHub, ArgoCD supprime la ressource correspondante dans le cluster Kubernetes pour maintenir une stricte équivalence.

13.4 Bilan

Le déploiement et le cycle de vie de l'application WokMaster sont désormais 100 % traçables, réversibles (***rollback*** en un clic) et gérés sous le paradigme GitOps.

Chapitre 14

Auto-guérison AIOps avec l'Agent MCP Custom (Model Context Protocol)

14.1 Objectif

Développer un agent d'auto-guérison autonome basé sur le protocole **Model Context Protocol (MCP)** et le modèle **Google Gemini 3.6-Flash**, capable de diagnostiquer les pannes de sécurité du cluster et d'exécuter des remédiations de façon contrôlée sous la supervision d'un opérateur humain (**Human-in-the-Loop**).

14.2 Architecture et Protocole MCP

Le serveur MCP (`mcp_k8s_server.py`) s'exécute en sous-processus et expose 3 outils d'administration Kubernetes sur l'entrée/sortie standard (`stdio`) :

1. `list_pods` : Extrait la liste des pods et leurs états fins (`CrashLoopBackOff`, `ErrImagePull`, `Running`).
2. `get_pod_logs` : Récupère les 50 dernières lignes de journaux de conteneurs pour l'analyse des anomalies.
3. `restart_deployment` : Déclenche un redémarrage progressif (`rollout restart`) d'un déploiement.

14.3 Mise en œuvre

14.3.1 Étape 1 — Code du Serveur MCP (`mcp_k8s_server.py`)

Le serveur JSON-RPC est implanté dans le répertoire `$HOME/kep-aiops/` :

```

1 import sys, json, subprocess
2
3 def list_pods(namespace="default"):
4     try:
5         cmd = ["kubectl", "get", "pods", "-n", namespace, "-o", "json"]
6         res = subprocess.run(cmd, capture_output=True, text=True, check=
7             True)
8         data = json.loads(res.stdout)
9         pods = []
10        for item in data.get("items", []):
11            name = item["metadata"]["name"]
12            status = item["status"]["phase"]
13            container_statuses = item["status"].get("containerStatuses",
14                [])
15            for c_status in container_statuses:
16                state = c_status.get("state", {})
17                for state_name, state_details in state.items():
18                    if state_name in ["waiting", "terminated"]:
19                        status = state_details.get("reason", status)
20                pods.append({"name": name, "status": status})
21        return json.dumps({"status": "success", "pods": pods})
22    except Exception as e:
23        return json.dumps({"status": "error", "message": str(e)})
24
25 def get_pod_logs(pod_name, namespace="default"):
26     try:
27         cmd = ["kubectl", "logs", pod_name, "-n", namespace, "--tail=50"
28             ]
29         res = subprocess.run(cmd, capture_output=True, text=True, check=
30             True)
31         return json.dumps({"status": "success", "logs": res.stdout})
32    except Exception as e:
33        return json.dumps({"status": "error", "message": str(e)})
34
35 def restart_deployment(name, namespace="default"):
36     try:
37         cmd = ["kubectl", "rollout", "restart", f"deployment/{name}", "-
38             n", namespace]
39         res = subprocess.run(cmd, capture_output=True, text=True, check=
40             True)
41         return json.dumps({"status": "success", "message": f"Rollout
42             restart triggered for deployment/{name}"})
43    except Exception as e:
44        return json.dumps({"status": "error", "message": str(e)})
45
46 TOOLS = {

```

```

40     "list_pods": list_pods,
41     "get_pod_logs": get_pod_logs,
42     "restart_deployment": restart_deployment
43 }
44
45 def main():
46     print("Kubernetes MCP Server initialized.", file=sys.stderr)
47     while True:
48         try:
49             line = sys.stdin.readline()
50             if not line: break
51             req = json.loads(line)
52             method = req.get("method")
53             params = req.get("params", {})
54             req_id = req.get("id")
55
56             if method == "tools/list":
57                 resp = {"jsonrpc": "2.0", "id": req_id, "result": {"
tools": [
58                     {"name": "list_pods", "description": "List all pods
and statuses.", "inputSchema": {"type": "object", "properties": {"
namespace": {"type": "string"}}}},
59                     {"name": "get_pod_logs", "description": "Fetch
container logs.", "inputSchema": {"type": "object", "required": ["
pod_name"], "properties": {"pod_name": {"type": "string"}}}},
60                     {"name": "restart_deployment", "description": "
Trigger rollout restart.", "inputSchema": {"type": "object", "
required": ["name"], "properties": {"name": {"type": "string"}}}}
61                 ]}}
62             elif method == "tools/call":
63                 name = params.get("name")
64                 args = params.get("arguments", {})
65                 res = TOOLS[name](**args) if name in TOOLS else "Error"
66                 resp = {"jsonrpc": "2.0", "id": req_id, "result": {"
content": [{"type": "text", "text": res}]}
67                 sys.stdout.write(json.dumps(resp) + "\n")
68                 sys.stdout.flush()
69             except Exception as e:
70                 pass
71
72 if __name__ == "__main__":
73     main()

```

Listing 14.1 – Serveur MCP Kubernetes (mcp_k8s_server.py)

14.3.2 Étape 2 — Code de l'Agent Client Gemini MCP (mcp_agent_run.py)

L'agent client gère la boucle conversationnelle et impose le prompt de confirmation de sécurité :

```
1 import sys, json, subprocess, urllib.request, os
2
3 GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")
4 MODEL_NAME = "gemini-3.6-flash"
5
6 server_process = subprocess.Popen(
7     ["python3", "mcp_k8s_server.py"],
8     stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess.
9     PIPE,
10    text=True, bufsize=1
11 )
12
13 def call_mcp(method, params=None, req_id=1):
14     payload = {"jsonrpc": "2.0", "method": method, "params": params or
15     {}, "id": req_id}
16     server_process.stdin.write(json.dumps(payload) + "\n")
17     server_process.stdin.flush()
18     return json.loads(server_process.stdout.readline().strip())
19
20 # Interception de l'outil modificateur et invite Human-in-the-Loop (HITL
21 ) :
22 # if tool_name == "restart_deployment":
23 #     approval = input("\n[MCP SECURITY] L'agent souhaite ex cuter
24 #     restart_deployment. Autoriser ? (O/N) : ")
```

Listing 14.2 – Agent Client MCP Gemini avec contrôle humain (mcp_agent_run.py)

14.3.3 Démonstration de la Boucle d'Auto-guérison en Direct

```

farah@k8s-master:~/kep-aio$ python3 mcp_agent_run.py spring-backend-6db8c77df5-n4jmb
[System] MCP Server Subprocess launched: Kubernetes MCP Server initialized. Ready for stdio messages.

[Agent Prompt]: spring-backend-6db8c77df5-n4jmb

[Loop 1] Sending request to Gemini...
[Gemini Agent Decision] Call tool: get_pod_logs({"pod_name": "spring-backend-6db8c77df5-n4jmb"})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "error", "message": "Command '['kubect', 'logs', 'spring-backend-6db8c77df5-n4jmb', '-n', 'default', '--tail=50', '--all-co
ntainers']' returned non-zero exit status 1."}...
[Loop 2] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}...
[Loop 3] Sending request to Gemini...
[Gemini Agent Decision] Call tool: get_pod_logs({"pod_name": "spring-backend-59fd4df595-2jqtm"})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "success", "logs": "=== Vault Agent started! Log data will stream in below:\n\n=== Vault Agent configuration:\n\n
Api Address 1: http://bufconn\n
Cgo: disabled..."}
[Loop 4] Sending request to Gemini...

```

FIGURE 14.1 – Lancement de l'agent MCP : Gemini décide d'exécuter les outils `get_pod_logs` et `list_pods` pour isoler la cause du blocage (Vault Agent)

```

[Loop 1] Sending request to Gemini...
[Gemini Agent Decision] Call tool: get_pod_logs({"pod_name": "spring-backend-6db8c77df5-n4jmb"})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "error", "message": "Command '['kubectl', 'logs', 'spring-backend-6db8c77df5-n4jmb', '-n', 'default', '--tail=50', '--all-co
ntainers']' returned non-zero exit status 1."}...
[Loop 2] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}...
[Loop 3] Sending request to Gemini...
[Gemini Agent Decision] Call tool: get_pod_logs({"pod_name": "spring-backend-59fd4df595-2jqtm"})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "success", "logs": "=== Vault Agent started! Log data will stream in below:\n\n=== Vault Agent configuration:\n\n
Api Address 1: http://bufconn\n
Cgo: disabled..."}
[Loop 4] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}...
[Loop 5] Sending request to Gemini...

```

FIGURE 14.2 – Poursuite de l'analyse (Loop 4) et décision d'invoquer la fonction `restart_deployment` (Loop 5)

```

farah@k8s-master: ~/kep-alo
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "error", "message": "Command '['kubect', 'logs', 'spring-backend-6db8c77df5-n4jmb', '-n', 'default', '--tail=50', '--all-co
ntainers']' returned non-zero exit status 1."}...
[Loop 2] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}
[Loop 3] Sending request to Gemini...
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "success", "logs": "\tat org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolder
Filter.java:82) ~[spring-security-web-6.2.4.jar!/6.2.4]\n\tat or..."}
[Loop 4] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}
[Loop 5] Sending request to Gemini...
[Gemini Agent Decision] Call tool: restart_deployment({"name": "spring-backend"})
[MCP SECURITY] L'agent souhaite exécuter : restart_deployment({"name": "spring-backend"}).
Autoriser cette action ? (O/N) : |

```

FIGURE 14.3 – Interception de sécurité : L’agent marque une pause et sollicite l’accord explicite de l’opérateur humain ([MCP SECURITY] Autoriser cette action ? (O/N) :)

```

farah@k8s-master: ~/kep-alo
[Loop 3] Sending request to Gemini...
[Gemini Agent Decision] Call tool: get_pod_logs({"pod_name": "spring-backend-59fd4df595-2jqtm"})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "success", "logs": "\tat org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolder
Filter.java:82) ~[spring-security-web-6.2.4.jar!/6.2.4]\n\tat or..."}
[Loop 4] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}
[Loop 5] Sending request to Gemini...
[Gemini Agent Decision] Call tool: restart_deployment({"name": "spring-backend"})
[MCP SECURITY] L'agent souhaite exécuter : restart_deployment({"name": "spring-backend"}).
Autoriser cette action ? (O/N) : 0
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {"status": "success", "message": "Rollout restart triggered for deployment/spring-backend"}...
[Loop 6] Sending request to Gemini...
[Gemini Agent Decision] Call tool: list_pods({})
[MCP Client] Forwarding call to local MCP Server...
[MCP Server] Output: {
  "status": "success",
  "pods": [
    {
      "name": "angular-frontend-7f4559c9bf-2wg2k",
      "status": "Running"
    },
    {
      "name": "filebeat-filebeat-fjc9m",
      "status": "Running"
    }
  ]
}
farah@k8s-master: ~/kep-alo$ |

```

FIGURE 14.4 – Validation par la touche 0 : Exécution du redémarrage du déploiement et vérification du retour des pods au statut Running

14.4 Explications techniques

- **Decoupled Architecture** : La séparation entre le serveur MCP (`mcp_k8s_server.py`) et l’agent client (`mcp_agent_run.py`) garantit qu’aucun privilège Kubernetes n’est exposé directement à l’API cloud Gemini.
- **Human-in-the-Loop (HITL)** : Pour prévenir toute destruction accidentelle de ressources par une décision erronée de l’IA, toute action modificatrice du cluster (`restart_deployment`) est soumise à la saisie manuelle d’une confirmation de

sécurité (0/N).

14.5 Bilan

La boucle AIOps MCP apporte une capacité d’auto-remédiation autonome inédite qui sécurise l’exploitation sans jamais compromettre la souveraineté de l’opérateur humain.

Chapitre 15

Synthèse et Gestion de Projet (Sprints Agile sur 8 Semaines)

15.1 Objectif

Présenter l'organisation temporelle des travaux menés durant le stage d'été de 8 semaines (2 mois) au sein d'***ESPRIT***, selon la méthodologie ***Agile Scrum***, et démontrer la synergie globale des 12 briques technologiques de la plateforme KEP.

15.2 Planification des Sprints (Chrono-Stage sur 8 Semaines)

Le projet a été découpé en ***4 Sprints de 2 semaines*** chacun, garantissant des livrables fonctionnels et testés à la fin de chaque étape :

TABLE 15.1 – Découpage du projet KEP en Sprints Agile Scrum (8 semaines / 2 mois)

Sprint	Période	Objectifs & Livrables Majeurs
Sprint 1	Semaines 1 & 2	Étude & Provisionnement IaaS : Étude des besoins, architecture 3 VMs, scripts PowerShell REST API et écriture du code IaC Terraform pour le clonage sous VMware.
Sprint 2	Semaines 3 & 4	Configuration & Cluster K8s : Écriture des playbooks Ansible , préparation du noyau Linux, installation du moteur containerd et initialisation du cluster K8s avec le CNI Calico .
Sprint 3	Semaines 5 & 6	Sécurité Zéro-Trust & Observabilité : Déploiement de HashiCorp Vault, injection des secrets en RAM tmpfs , et déploiement de la pile ELK (Elasticsearch, Kibana, Filebeat, Metricbeat).
Sprint 4	Semaines 7 & 8	GitOps, Alerting & AIOps : Mise en place d'ArgoCD, canal d'alerte Discord via Kubewatch, diagnostic K8sGPT et développement de l'Agent MCP d'auto-guérison (Gemini 3.6-Flash).

15.3 Matrice de Synthèse d'Intégration

TABLE 15.2 – Matrice d'interconnexion globale des composants de la plateforme KEP

Étape / Phase	Outils Impliqués	Rôle & Synergie
Jour 0 : Provisionnement	Terraform, VMware API	Clonage automatisé des 3 VMs.
Jour 1 : Configuration	Ansible, Ansible Vault	Installation idempotente des paquets K8s.
Jour 2 : Sécurité	HashiCorp Vault, tmpfs	Injection Zéro-Trust des secrets en RAM.
Jour 2 : Observabilité	ELK, Filebeat, Metricbeat	Centralisation des logs et métriques.
Jour 2 : GitOps	ArgoCD, GitHub	Continuous Delivery et anti-dérive.
Jour 2 : Alerting	Kubewatch, Discord	Capture des pannes et alerte instantanée.
Jour 2 : AIOps	K8sGPT, Agent MCP, Gemini	Diagnostic IA et auto-guérison guidée.

15.4 Bilan de l'Intégration

L'articulation entre l'Infrastructure as Code (Terraform/Ansible), le paradigme GitOps (ArgoCD), la sécurité Zéro-Trust (Vault), l'observabilité (ELK) et l'AIOps (K8sGPT / Agent MCP) confère à la plateforme une résilience complète et une autonomie opérationnelle unique.

—

Conclusion Générale et Perspectives

Conclusion

Ce travail réalisé lors de mon stage d'été au sein d'***ESPRIT** sous l'encadrement du **Pr. Omar FITOURI** a permis de mener à bien la conception, le déploiement et l'automatisation complète de la plateforme d'entreprise **KEP (Kubernetes Enterprise Platform)**.

L'infrastructure réalisée démontre qu'il est possible de concilier une sécurité Zéro-Trust stricte (secrets injectés uniquement en mémoire vive RAM `tmpfs`), une automatisation totale par le code (Terraform, Ansible, GitOps ArgoCD), une observabilité centralisée (ELK) et une résilience intelligente propulsée par l'Intelligence Artificielle (K8sGPT, Agent MCP Gemini).

Perspectives d'avenir

Les perspectives d'évolution de la plateforme KEP incluent :

1. **L'auto-scaling prédictif** : Ajuster dynamiquement le nombre de répliques de pods en fonction des prédictions de charge anticipées par l'IA.
2. **L'intégration de la sécurité des images (Trivy)** : Étendre les outils du serveur MCP pour analyser la sécurité des images de conteneurs avant tout redémarrage correctif.

Annexe A

Annexes

A.1 Annexe A : Fichier d’inventaire Ansible (‘hosts.ini’)

```
1 [master]
2 k8s-master ansible_host=192.168.233.188
3
4 [worker]
5 k8s-worker ansible_host=192.168.233.190
6
7 [elk]
8 elk-vault ansible_host=192.168.233.189
9
10 [all:vars]
11 ansible_user=farah
12 ansible_ssh_pass="{{ vault_ssh_pass }}"
13 ansible_become_pass="{{ vault_become_pass }}"
14 ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

A.2 Annexe B : Code du Serveur MCP (‘mcp_{k8s}server.py’)

```
1 # Serveur MCP JSON-RPC Kubernetes pour l’agentic SRE AIOps
2 # Expose list_pods, get_pod_logs et restart_deployment sur stdio
3 # (Voir Chapitre 14 pour l’implémentation intégrale)
```

Webographie et Ressources Techniques

Documentation Officielle des Projets CNCF & Orchestration

1. **Kubernetes Documentation** : Documentation officielle du moteur d'orchestration de conteneurs.
<https://kubernetes.io/docs/>
2. **Calico CNI Documentation** : Plugin de réseau virtuel et de sécurité inter-pods.
<https://docs.tigera.io/calico/latest/about/>
3. **Helm Package Manager** : Gestionnaire de paquets applicatifs pour Kubernetes.
<https://helm.sh/docs/>

Infrastructure as Code & Gestion de Configuration

1. **Terraform by HashiCorp** : Documentation du provisionneur d'infrastructure IaC.
<https://developer.hashicorp.com/terraform/docs>
2. **Ansible & Ansible Vault** : Documentation du moteur de configuration automatisée et du coffre de variables.
<https://docs.ansible.com/>
3. **VMware Workstation REST API** : Spécifications de l'interface REST `vmrest.exe`.
<https://developer.broadcom.com/>

Sécurité Zéro-Trust & Gestion des Secrets

1. **HashiCorp Vault Documentation** : Gestionnaire centralisé de secrets et d'injection en mémoire RAM `tmpfs`.
<https://developer.hashicorp.com/vault/docs>
2. **Vault Agent Sidecar Injector** : Spécifications des annotations d'injection automatique.

<https://developer.hashicorp.com/vault/docs/platform/k8s/injector>

Observabilité & Centralisation des Journaux

1. **Elasticsearch & Kibana 8.x** : Guide officiel de la pile d'observabilité ELK.
<https://www.elastic.co/guide/index.html>
2. **Bitnami Kubewatch** : Agent de surveillance des événements Kubernetes et d'alerting Discord.
<https://github.com/bitnami-labs/kubewatch>

GitOps & Intelligence Artificielle (AIOps)

1. **ArgoCD GitOps Documentation** : Moteur de synchronisation continue et d'anti-dérive.
<https://argo-cd.readthedocs.io/>
2. **K8sGPT CNCF Project** : Outil de diagnostic automatisé assisté par IA.
<https://k8sgpt.ai/>
3. **Model Context Protocol (MCP)** : Spécifications du protocole standard d'intégration d'outils pour LLMs.
<https://modelcontextprotocol.io/>
4. **Google Gemini API & AI Studio** : Documentation du modèle LLM gemini-3.6-flash.
<https://ai.google.dev/docs>